Rapport de stage de fin d'année

ISTY – IATIC4 – Année universitaire 2025–2026

Calculs de similarité entre exercices de programmation informatique

pour la recommandation et la substitution d'exercices dans les EIAH

Juvenal MALECOU NGUIMBI

Structure d'accueil :	Laboratoire DICEN-IdF – Université Gustave Eiffel
Lieu :	Campus de Marne-la-Vallée, 5 bd Descartes, 77454 Champs-sur-Marne
Dates du stage :	du 08/06/2026 au 05/11/2026
Maître de stage :	M. Olivier CHAMPALLE
Tuteur pédagogique (ISTY) :	Mme Laurence PILARD
Confidentialité :	OUI

Remerciements

Je tiens tout d'abord à remercier chaleureusement Monsieur Olivier Champalle, mon maître de stage au sein du laboratoire DICEN-IdF, pour la confiance qu'il m'a accordée en m'intégrant à ce projet de recherche. Ses conseils avisés, sa disponibilité et son encadrement scientifique ont été déterminants dans la structuration de ma démarche et m'ont permis d'approfondir considérablement mes connaissances sur le sujet.

Mes remerciements s'adressent également à Madame Laurence Pilard, ma tutrice pédagogique à l'ISTY, pour son suivi attentif tout au long de cette période, ses conseils et son accompagnement.

Je souhaite aussi exprimer ma gratitude envers Adel Ziani, dont les travaux préliminaires ont constitué un point de départ précieux pour mes propres expérimentations.

Je remercie plus largement l'ensemble des membres de l'équipe PLaTon et de l'Université Gustave Eiffel pour leur accueil et pour avoir créé un environnement de travail à la fois stimulant et convivial au quotidien.

Enfin, je remercie l'équipe pédagogique de l'ISTY pour la qualité de la formation d'ingénieur dispensée, ainsi que ma famille et mes proches pour leur soutien indéfectible tout au long de mes études.

Résumé

Ce rapport étudie la substituabilité entre exercices de programmation, c'est-à-dire la possibilité d'identifier des activités qui, sans être identiques, mobilisent des connaissances, des structures de contrôle et des démarches algorithmiques suffisamment proches pour constituer des alternatives pertinentes.

Les objectifs sont multiples : aider un enseignant à retrouver, dans une banque d'exercices, une activité proche d'un problème existant ; recommander à un étudiant un entraînement sollicitant une démarche comparable, sans lui faire répéter exactement la même consigne. La substituabilité ne doit toutefois pas être confondue avec la similarité textuelle, la détection de plagiat ou l'équivalence fonctionnelle de deux programmes, même si ces domaines partagent certaines méthodes. Deux exercices peuvent traiter des problèmes différents tout en mobilisant des stratégies de résolution proches. À l'inverse, deux énoncés utilisant un vocabulaire semblable peuvent nécessiter des démarches distinctes.

Dans le contexte étudié, aucune paire d'exercices n'est annotée selon son degré de substituabilité et aucune trace d'activité ou de performance d'apprenants n'est disponible. L'étude s'appuie donc principalement sur les propriétés intrinsèques des corrections de référence. Des expérimentations préliminaires sur les énoncés, fondées sur TF–IDF et SBERT, ont fourni un signal de proximité thématique. Les ambiguïtés, la variabilité des formulations et la dépendance de certaines consignes à un contexte implicite conduisent cependant à retenir le code comme source principale d'information.

Parmi les approches proposées dans la littérature sur des sujets voisins, trois ont retenu notre attention ; elles extraient trois caractéristiques des corrections : une similarité par distance d'édition entre arbres syntaxiques abstraits normalisés (TSED), une similarité globale fondée sur des métriques structurelles (EIIA), et une similarité calculée à partir d'un *Normalized Control Structure Tree* (NCST), une représentation construite au cours de ce stage. Construit à partir de l'arbre syntaxique concret produit par Tree-sitter (un analyseur syntaxique, ou *parseur*, incrémental), le NCST est normalisé afin de conserver l'organisation des structures de contrôle, notamment les conditions, les itérations et les transferts, tout en réduisant l'influence des détails syntaxiques et des styles d'écriture.

Les méthodes sont évaluées sur des variantes contrôlées de programmes, puis sur un corpus plus large. Elles produisent des indicateurs continus et interprétables permettant de filtrer des paires candidates à une validation pédagogique humaine, nécessaire pour déterminer si la proximité structurelle observée correspond effectivement à une substituabilité du point de vue de l'apprentissage.

Mots-clés : EIAH, similarité de code, normalisation de code, exercices de programmation, arbre syntaxique abstrait, distance d'édition, structures de contrôle.

Abstract

This report studies the substitutability between programming exercises, that is, the possibility of identifying activities that, without being identical, involve sufficiently similar knowledge, control structures, and algorithmic approaches to constitute relevant alternatives.

The objectives are multiple : to help a teacher find, in an exercise bank, an activity close to an existing problem ; to recommend to a student a training exercise involving a comparable approach, without making them repeat exactly the same problem, etc. Substitutability should not, however, be confused with textual similarity, plagiarism detection, or the functional equivalence of two programs. Although the methods used share some common aspects, two exercises can address different problems while mobilizing similar solution strategies. Conversely, two exercise statements using similar vocabulary may require distinct approaches.

In the context studied, no pair of exercises is annotated according to its degree of substitutability, and no learner activity or performance data are available. The study therefore relies mainly on the intrinsic properties of the reference solutions. Preliminary experiments on the exercise statements, based on TF–IDF and SBERT, provided a signal of thematic proximity. However, the ambiguities, variability of formulations, and dependence of some statements on implicit context lead us to use the code as the main source of information.

Among the diversity of approaches found in the literature on related topics, three approaches caught our attention, through which we can extract three characteristics from the reference solutions : a similarity measure based on the edit distance between normalized abstract syntax trees (TSED), a global similarity measure based on structural metrics (EIIA), and a similarity measure calculated from a *Normalized Control Structure Tree* (NCST), which is a construction developed as part of our work.

The NCST is constructed from the concrete syntax tree produced by Tree-sitter (a tool for generating parsers and an incremental parsing library, or more simply, a parser). The NCST is normalized in order to preserve the organization of the control structures, particularly conditions, iterations, and transfers, while reducing the influence of syntactic details and programming styles.

The methods are evaluated on controlled variants of programs, and then on a larger corpus. They produce continuous and interpretable indicators that make it possible to filter candidate pairs for human pedagogical validation, which is necessary to determine whether the observed structural proximity effectively corresponds to substitutability from a learning perspective.

Keywords : EIAH, code similarity, code normalization, programming exercises, abstract syntax tree, edit distance, control structures.

Table des matières

Remerciements **i**

Résumé **ii**

Lexique **ix**

Introduction générale **1**

1 Présentation des structures d'accueil **2**

1.1 L'Université Gustave Eiffel 2

1.2 Le laboratoire DICEN-IdF 3

1.3 L'Institut Gaspard Monge 3

2 Contexte et organisation du stage **4**

2.1 Description du projet 4

2.2 Logistique du stage 5

2.3 Méthodologie 7

3 État de l'art et positionnement **8**

3.1 Qu'est-ce que la similarité ? 8

3.2 Domaine et approches des travaux sur les similarités dans les EIAH 10

3.3 Quels sont les outils et méthodes utilisés par ces approches ? 11

3.4 Discussion sur les limites des méthodes 12

3.5 Pistes explorées et écartées 14

4 Ma proposition **16**

4.1 Architecture globale du pipeline 16

4.2 Évaluation de l'information indépendante de l'arbre (EIIA) 18

4.3 AST normalisé et distance TSED 19

4.3.1 Conversion du CST en AST simplifié 19

4.3.2 Normalisation de l'AST 19

4.3.3 Distance d'édition entre arbres 20
4.4 Arbre structurel normalisé (NCST) 21
4.4.1 Construction et normalisation du NCST 21
4.4.2 Exemple de NCST normalisé et linéarisation 21
4.4.3 Invariance du squelette structurel 22
4.4.4 Algorithme d'appariement naïf ordonné 22
4.4.5 Calcul de la similarité NCST 24
4.4.6 Positionnement par rapport au TSED et à l'EIIA 24
4.5 Exécution et synthèse des mesures 24
4.6 Interface de comparaison Streamlit 25
4.7 Protocole d'évaluation des approches 26

5 Résultats expérimentaux **27**
5.1 Constitution du corpus et variantes contrôlées 27
5.2 Résultats sur les variantes contrôlées 28
5.2.1 Bilan sur les variantes contrôlées 30
5.2.2 Fusion des trois approches 30
5.3 Résultats sur le corpus complet 31
5.3.1 Sous-corpus analysé 31
5.3.2 Distribution des scores 31
5.3.3 Accord entre les mesures 32
5.3.4 Paires candidates 33
5.3.5 Divergences entre les mesures 33
5.4 Synthèse comparative 34

6 Discussion sur les limites et perspectives **36**
6.1 Limites techniques des méthodes 36
6.2 Limites de l'évaluation 36
6.3 Difficultés rencontrées 37
6.4 Perspectives 38

Conclusion générale **40**

Bibliographie **42**
Webographie 44

Table des figures

1.1 Regroupement des établissements formant l'Université Gustave Eiffel 2

2.1 Acteurs du stage et environnement de travail 6
2.2 Diagramme de Gantt du stage 6
2.3 Démarche de travail suivie pendant le stage 7

3.1 Les quatre grandes finalités de la mesure de similarité dans les EIAH (orientations divergentes à partir d'un même besoin technique). 10
3.2 Familles de représentations mobilisées pour comparer des exercices de programmation 13
3.3 Cartographie des limites : pourquoi chaque approche individuelle échoue à caractériser seule la substituabilité pédagogique. 15

4.1 Architecture globale du pipeline d'évaluation de la similarité multi-niveaux. L'EIIA et le TSED partagent l'AST normalisé ; le NCST est construit directement à partir du CST. L'interface Streamlit exploite le CSV produit par le pipeline. 17
4.2 Chaîne de calcul de l'EIIA : extraction de deux profils structurels, puis distance euclidienne normalisée. 19
4.3 Normalisation d'une instruction. Les nœuds en rouge portent des détails de surface (noms, valeurs) ; après normalisation, ils sont remplacés par des formes canoniques (en vert). Les opérateurs et la forme de l'affectation sont conservés. 20
4.4 Exemple de calcul TSED sur deux AST normalisés de quatre nœuds. 21
4.5 NCST normalisé d'une fonction (gauche), avec entre crochets la relation qui relie chaque nœud à son parent, et liste (*kind*, *level*) obtenue par parcours en profondeur (droite). Le retrait traduit la profondeur dans l'arbre, tandis que le niveau n'augmente que sur les arêtes `BODY` et `THEN`. 22
4.6 Invariance du NCST normalisé sur deux programmes différents. Les codes A et B utilisent des variables, des opérateurs et des valeurs de retour distincts, mais leur normalisation produit la même liste de nœuds abstraits. 23
4.7 Appariement ordonné du petit arbre vers le grand arbre. Les pointillés relient des nœuds de même *kind* et de même *level*, dans un ordre croissant d'indices. 23
4.8 Fonctionnement de l'interface Streamlit : préparation du jeu de fichiers, appel du pipeline, pondération facultative et restitution des scores et des arbres. 25
4.9 Protocole d'évaluation en deux phases. 26

5.1 Correction de référence et variante contrôlée « `while` avec index » : même stratégie, structure de parcours différente. 28
5.2 Répartition des scores par intervalles de 0,1 sur le sous-corpus GitHub (4 851 paires). 32

Liste des tableaux

3.1 Pistes explorées et raisons de leur abandon. 15

4.1 Synthèse des trois niveaux de représentation du pipeline. 17
4.2 Composantes du profil structurel utilisé par l'EIIA. 18
4.3 Structure du fichier CSV exporté par le pipeline : une ligne par paire, une colonne par mesure, et une colonne d'erreur en cas d'échec de l'analyse. 24
4.4 Exemples de profils de scores et de leur interprétation. 25

5.1 Statistiques descriptives du corpus de corrections de référence. 28
5.2 Statistiques descriptives du jeu de variantes contrôlées. 28
5.3 Variantes contrôlées dérivées de la correction de référence. 29
5.4 Scores obtenus entre la correction de référence `01_base.py` et chacune de ses douze variantes contrôlées. 29
5.5 Scores pondérés à poids égaux entre la correction de référence et chacune de ses variantes. 31
5.6 Distribution des scores sur les 4 851 paires du sous-corpus GitHub. Les deux dernières colonnes donnent un nombre de paires. 32
5.7 Corrélation de rang de Spearman entre les trois scores (4 851 paires). 32
5.8 Score moyen selon la présence de fonctions dans les deux fichiers. 33
5.9 Synthèse du comportement des trois mesures sur les deux phases d'évaluation. . . . 34

Lexique

Les termes sont classés par ordre alphabétique. Les sigles anglais sont conservés lorsqu'ils sont d'usage dans la littérature.

Apprenant
Personne qui suit une formation ; dans ce rapport, l'étudiant qui réalise les exercices de programmation.

AST (*Abstract Syntax Tree*, arbre syntaxique abstrait)
Représentation arborescente d'un programme dans laquelle chaque nœud correspond à une construction du langage (boucle, condition, affectation, appel de fonction), indépendamment de la mise en forme du texte.

CFG (*Control Flow Graph*, graphe de flot de contrôle)
Graphe dont les nœuds sont des instructions et dont les arcs indiquent l'ordre possible d'exécution.

Clone de code
Fragment de code identique ou très proche d'un autre fragment ; la détection de clones cherche à les repérer dans un logiciel.

CMGA
Méthode de détection de code sémantiquement similaire combinant un filtrage par métriques et une comparaison de graphes de dépendances normalisés [23].

Cold start (démarrage à froid)
Situation dans laquelle un système doit fonctionner sans données historiques (annotations, résultats d'apprenants) sur lesquelles s'appuyer.

Corpus
Ensemble des exercices (énoncés et corrections) utilisés pour les expérimentations.

Corrélation de Spearman
Coefficient compris entre -1 et 1 qui indique si deux mesures classent des éléments dans le même ordre : chaque score est remplacé par son rang, puis on calcule la corrélation entre les rangs. Il vaut 1 lorsque les deux classements sont identiques et 0 lorsqu'ils sont sans lien.

CPG (*Code Property Graph*)
Graphe unique, produit par l'outil Joern, qui superpose l'AST, le CFG et les dépendances d'un programme.

CSSG
Mesure de similarité de code fondée sur des graphes sémantiques [10].

Distance d'édition
Coût minimal des opérations élémentaires (insertion, suppression, substitution) nécessaires pour transformer un objet en un autre ; s'applique aux chaînes, aux arbres (TED) et aux graphes (GED).

EIAH
Environnement informatique pour l'apprentissage humain : logiciel ou plateforme conçu pour soutenir l'apprentissage, par exemple un exerciseur de programmation.

Embedding (plongement vectoriel)
Représentation d'un texte ou d'un code sous forme de vecteur de nombres, telle que deux éléments proches en sens soient proches dans l'espace vectoriel.

Exerciseur
EIAH proposant des exercices à correction automatique pour l'entraînement.

GED (*Graph Edit Distance*)
Distance d'édition entre graphes.

Joern
Outil d'analyse statique de code, multi-langage, produisant un Code Property Graph.

Normalisation de code
Ensemble de transformations qui ramènent des écritures différentes mais équivalentes d'un programme à une forme commune (renommage canonique, unification des boucles, suppression des commentaires, etc.).

NCST
Normalized Control Structure Tree : arbre normalisé des structures de contrôle, construit à partir du CST Tree-sitter et utilisé pour comparer l'organisation des fonctions, séquences, boucles, conditions, exceptions et transferts de contrôle.

EIIA
Indicateur global calculé à partir des métriques structurelles extraites des AST normalisés, utilisé pour estimer et comparer la complexité structurelle de deux corrections.

PDG (*Program Dependence Graph*)
Graphe qui réunit, pour une fonction, les dépendances de contrôle et les dépendances de données entre instructions.

Recommandation d'exercices
Proposition automatique à un apprenant d'un exercice adapté à sa progression.

SBERT (*Sentence-BERT*)
Modèle de langage qui transforme une phrase en vecteur afin de comparer des phrases par leur sens [19].

Substituabilité
Propriété de deux exercices qui peuvent être échangés dans un parcours d'apprentissage parce qu'ils mobilisent des notions et une démarche de résolution proches.

TED (*Tree Edit Distance*)
Distance d'édition entre arbres ordonnés ; l'algorithme de Zhang et Shasha en est le calcul de référence [25].

Token (lexème)
: Unité lexicale élémentaire d'un programme (mot-clé, identifiant, opérateur, littéral) produite par l'analyse lexicale.

TSED
: Similarité de code obtenue en normalisant la distance d'édition entre AST [20].

Variante contrôlée
: Version d'un programme de référence obtenue par une transformation unique et identifiée (renommage, changement de boucle, extraction de fonction), utilisée pour observer la réaction d'une mesure à cette transformation.

Variation de code
: Phénomène par lequel une même tâche peut être programmée de nombreuses façons syntaxiquement différentes mais sémantiquement équivalentes [23].

Vérité terrain
: Référence considérée comme exacte (par exemple un jugement d'enseignant) à laquelle on compare les résultats d'une méthode automatique.

Introduction générale

L'enseignement de la programmation repose largement sur la pratique : un étudiant progresse en résolvant des exercices, en se trompant puis en recommençant. Les enseignants s'appuient pour cela sur des environnements informatiques pour l'apprentissage humain (EIAH), comme la plateforme PLaTon développée à l'Université Gustave Eiffel, qui proposent des banques d'exercices corrigés automatiquement. À mesure que ces banques grandissent, une question devient centrale : comment retrouver, parmi des centaines d'exercices, ceux qui se ressemblent ? Un enseignant voudrait trouver une variante d'un exercice pour préparer une évaluation ; un étudiant qui a réussi un exercice gagnerait à s'entraîner sur un autre qui mobilise la même démarche, sans refaire exactement la même consigne.

C'est dans ce contexte que s'inscrit mon stage, réalisé du 8 juin au 5 novembre 2026 au sein du laboratoire DICEN-IdF, dans les locaux de l'Institut Gaspard Monge, sous l'encadrement de M. Olivier Champalle. Le sujet qui m'a été confié porte sur le calcul de similarité entre exercices de programmation, en vue de la recommandation et de la substitution d'exercices. La notion visée est la *substituabilité* : deux exercices sont substituables lorsque, sans être identiques, ils mobilisent des connaissances et une démarche de résolution suffisamment proches pour être échangés dans un parcours d'apprentissage. Cette notion se distingue de la ressemblance des énoncés, de la détection de plagiat et de l'équivalence fonctionnelle de deux programmes.

Le travail se heurte à une contrainte forte, dite de *démarrage à froid* : aucune paire d'exercices n'est annotée selon sa substituabilité et aucune trace d'activité d'apprenants n'est disponible. Il n'est donc pas possible d'apprendre la similarité à partir d'exemples ; elle doit être estimée directement à partir des exercices eux-mêmes. Après des essais préliminaires sur le texte des énoncés, j'ai retenu comme source principale d'information la correction de référence de chaque exercice, dont le code reflète la démarche attendue.

Ma mission comportait trois volets : réaliser un état de l'art sur la similarité entre exercices, algorithmes et codes ; concevoir et implémenter des mesures de similarité adaptées à ce contexte ; enfin, les valider sur des données expérimentales. Elle prolonge les travaux préliminaires d'Adel Ziani [26], qui avaient constitué la base d'exercices et exploré de premières approches. J'ai abouti à un pipeline qui calcule trois mesures complémentaires sur les corrections : l'EIIA, qui compare des profils de complexité ; le TSED, une distance d'édition entre arbres syntaxiques normalisés ; et le NCST, une représentation de l'organisation des structures de contrôle construite au cours du stage. Une interface permet de lancer les comparaisons et d'inspecter les représentations produites.

Ce rapport est organisé en six chapitres. Le chapitre 1 présente les structures d'accueil et le chapitre 2 le projet, l'organisation du stage et la démarche suivie. Le chapitre 3 dresse l'état de l'art et justifie le positionnement retenu. Le chapitre 4 décrit le pipeline et les trois mesures, puis le chapitre 5 présente leur évaluation sur des variantes contrôlées et sur un corpus d'exercices réels. Le chapitre 6 discute les limites du travail, les difficultés rencontrées et les perspectives, avant une conclusion générale. Les détails techniques sont rassemblés dans un rapport complémentaire, auquel le texte renvoie lorsque c'est utile.

Chapitre 1

Présentation des structures d'accueil

Dans ce chapitre, je présente l'Université Gustave Eiffel, au sein de laquelle s'est déroulé mon stage, le laboratoire DICEN-IdF, qui a financé ce stage, ainsi que l'Institut Gaspard Monge (IGM), où j'ai travaillé en étroite collaboration avec l'équipe PLaTon, chargée de la plateforme à laquelle les résultats de ces recherches sont principalement destinés.

1.1 L'Université Gustave Eiffel

L'Université Gustave Eiffel est une université française créée en janvier 2020 par la fusion de plusieurs établissements [29, 30] (figure 1.1) :

- **L'UPEM** : L'Université Paris-Est Marne-la-Vallée, fondée en 1991.
- **L'IFSTTAR** : L'Institut français des sciences et technologies des transports, de l'aménagement et des réseaux.
- **L'ESIEE Paris** : Une école d'ingénieurs située à la cité Descartes.
- **L'ENSG-Géomatique** : Une école d'ingénieurs spécialisée en géomatique.
- **L'EIVP** : Une école d'ingénieurs située à Paris.
- **L'Éav&t** : L'école d'architecture de la ville et des territoires Paris-Est.

L'UGE contient au total 16 composantes d'enseignement ainsi que 23 composantes de recherche. Parmi ses principales activités, on retrouve la formation, la recherche et l'innovation.

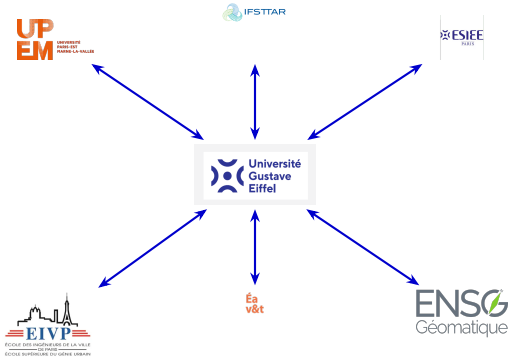


Figure 1.1 – Regroupement des établissements formant l'Université Gustave Eiffel

1.2 Le laboratoire DICEN-IdF

Le laboratoire DICEN-IdF (Dispositifs d'Information et de Communication à l'Ère Numérique – Paris Île-de-France) est une unité de recherche multi-tutelles. Ses axes de recherche portent sur l'ingénierie documentaire, les humanités numériques et l'intelligence collective [28, 27].

C'est à ce laboratoire qu'est rattaché mon maître de stage, Monsieur Olivier Champalle, et c'est également le DICEN-IdF qui finance ce stage. À ce titre, le laboratoire constitue le cadre scientifique principal du projet, en y apportant les fonds nécessaires ainsi que son expertise sur l'ingénierie des connaissances et l'analyse des traces d'apprentissage dans les environnements numériques.

1.3 L'Institut Gaspard Monge

Bien que mon stage soit financé par le DICEN-IdF, il s'est déroulé physiquement dans les locaux de l'Institut Gaspard Monge (IGM), situés sur le campus de l'Université Gustave Eiffel.

L'IGM est un institut reconnu pour ses travaux en informatique fondamentale et appliquée. C'est sur ce site que j'ai mené mes recherches au quotidien, ce qui m'a permis de travailler en étroite collaboration avec l'équipe PLaTon. PLaTon est un exerciseur développé et géré par cette équipe. Cette proximité géographique et technique était essentielle, car c'est au sein de cette plateforme pédagogique que les algorithmes de calcul de similarité ont vocation à être intégrés et exploités principalement.

Chapitre 2

Contexte et organisation du stage

Dans ce chapitre, je présente le projet dans lequel s'inscrit mon stage, puis la manière dont le stage a été organisé, et enfin la démarche que j'ai suivie pour mener ce travail de recherche.

2.1 Description du projet

Dans le cadre des enseignements d'algorithmique et de programmation informatique, les enseignants s'appuient de plus en plus sur des Environnements Informatiques pour l'Apprentissage Humain (EIAH), tels que la plateforme PLaTon. La base de cette plateforme est constituée d'une multitude d'exercices, chacun étant défini par un énoncé textuel et une correction de référence. L'idée fondatrice du projet est de comprendre s'il est possible d'identifier des exercices « proches » ou « distants » entre eux, quel que soit le langage de programmation, et de caractériser cette distance de manière à construire une classification réutilisable.

Cette question est complexe et soulève des enjeux sémantiques. Si la langue française dispose de nombreux termes pour l'exprimer (deux exercices peuvent être *analogues*, *comparables*, *semblables*, *voisins*, *équivalents* ou *substituables*), chaque mot porte une nuance différente. Pour évaluer la substituabilité pédagogique, plusieurs critères ont été identifiés :

— La **mobilisation des mêmes connaissances**, comme l'utilisation d'un même type de structure de données (liste, dictionnaire) ou d'une même notion algorithmique.
— Un **comportement** et une **complexité** similaires, tant dans l'architecture du code que dans le temps d'exécution.
— Des **résultats pédagogiques** similaires, garantissant que le travail de l'apprenant cible bien le même concept, même si le résultat brut de l'exécution diffère.
— Une **difficulté** (ou un effort de réalisation) comparable pour l'étudiant.

La classification recherchée doit prendre en compte les concepts et notions d'algorithmique nécessaires pour résoudre l'exercice, et les compétences propres au langage de programmation utilisé (Python, Java, C, etc.), avec leur niveau de difficulté. La définition précise de la similarité est approfondie dans l'état de l'art.

Une telle mesure de similarité ouvre plusieurs cas d'utilisation, à plus ou moins long terme, au sein des EIAH de type exerciseur.

Pour l'enseignant :

— Rechercher rapidement dans la base un exercice similaire à ceux qu'il possède déjà dans ses

fiches de TP, afin d'éviter la redondance.

— Se voir suggérer des variantes « proches » structurellement pour préparer des évaluations.

— Repartir d'un exercice existant pour le dériver à la marge.

— Catégoriser et étiqueter automatiquement la difficulté et les notions mobilisées dans la base d'exercices.

Pour l'étudiant :

— S'entraîner sur des exercices proches mais non identiques, afin de renforcer des notions sans refaire la même consigne.

— Bénéficier d'un système de recommandation adaptatif suggérant des exercices de difficulté équivalente.

Pour l'évaluation des IA :

— Faciliter la validation (ou l'invalidation) d'exercices générés par une intelligence artificielle, en comparant automatiquement leur structure avec le corpus validé par les enseignants.

Le projet fait face à une contrainte majeure dite de *démarrage à froid* (*cold start*). Aucune paire d'exercices n'est annotée formellement pour indiquer son degré de substituabilité, et aucune trace d'activité d'apprenants (temps passé, taux d'erreur) n'est disponible. L'analyse doit donc s'opérer de manière purement statique et non supervisée, en se fondant exclusivement sur les énoncés et l'unique correction de référence de chaque exercice.

Ainsi, l'objectif est de proposer des mesures automatiques de similarité entre exercices, utilisables sans données d'apprenants et interprétables par un enseignant, afin d'identifier des paires d'exercices candidates à la substitution. Pour y parvenir, les missions qui m'ont été confiées sont les suivantes :

— Réaliser un état de l'art sur la notion de similarité entre exercices, algorithmes et codes.

— M'appuyer sur le travail préliminaire d'Adel Ziani (stage de M1, 2025) [26], qui a permis de construire la base de données et d'explorer les premières approches textuelles.

— Créer, adapter et implémenter plusieurs méthodes de détection de similarité.

— Valider et calibrer ces méthodes sur des données expérimentales.

Les livrables attendus sont le code source des algorithmes, les jeux de données expérimentaux, ainsi que le présent rapport, qui comprend l'état de l'art.

2.2 Logistique du stage

Ce stage, d'une durée d'environ cinq mois, s'est déroulé du 8 juin au 5 novembre 2026. Bien que rattaché au laboratoire DICEN-IdF (Université Gustave Eiffel), le travail quotidien a été effectué dans les locaux de l'Institut Gaspard Monge (IGM), sur le campus de Marne-la-Vallée à Champs-sur-Marne.

L'encadrement scientifique a été assuré par mon maître de stage, Monsieur Olivier Champalle (DICEN-IdF), à l'origine du sujet de recherche. Au quotidien, j'ai travaillé à proximité de l'équipe PLaTon, ce qui me permettait d'échanger sur le fonctionnement de la plateforme et sur la structure des exercices. Le suivi pédagogique, garantissant l'adéquation du stage avec les exigences de ma formation à l'ISTY, a été assuré par Madame Laurence Pilard. La figure 2.1 résume l'organisation des acteurs autour du projet.

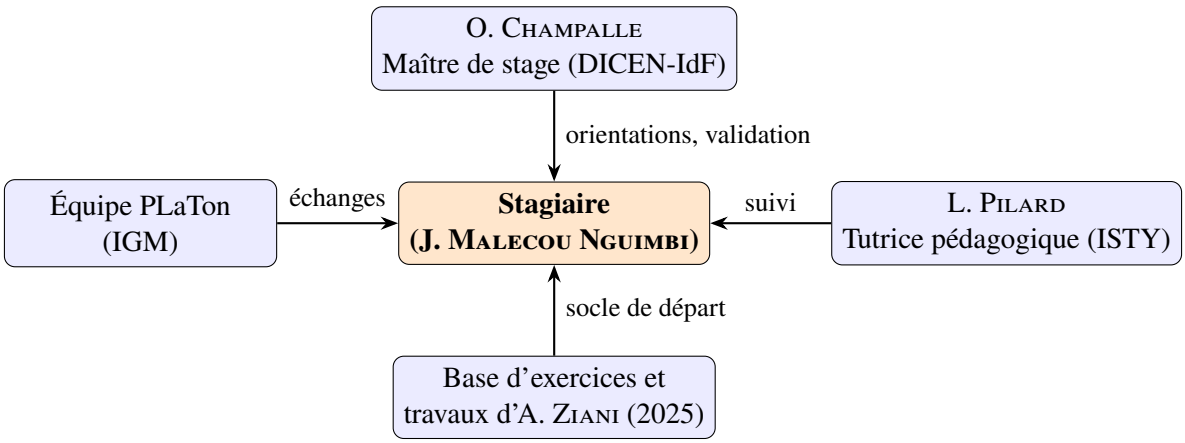


Figure 2.1 – Acteurs du stage et environnement de travail

Des points réguliers ont eu lieu avec mon maître de stage pour discuter des expérimentations, valider les choix techniques, comme le passage d'une similarité fondée sur les énoncés à une similarité fondée sur le code, et planifier la suite du travail. Les principaux outils utilisés sont les suivants :

- **Python** : langage principal pour l'ensemble des scripts d'expérimentation ;
- **Tree-sitter** : outil d'analyse syntaxique (*parser*) utilisé pour analyser le code source et produire des arbres syntaxiques, pour de nombreux langages de programmation ;
- **`scikit-learn` et `sentence-transformers`** : bibliothèques utilisées pour les expériences sur les énoncés (TF-IDF et SBERT) ;
- **Git** : gestion des versions du code ;
- **Google Scholar** : moteur de recherche d'articles scientifiques, utilisé pour l'état de l'art ;
- **Zotero** : stockage des articles de recherche et rédaction de leurs résumés ;
- **Google Drive** : partage des documents, des résultats d'expérimentation et des comptes rendus avec mon maître de stage ;
- **LaTeX** : rédaction du rapport.

La figure 2.2 présente le déroulement du stage. Les phases situées à gauche de la ligne « Aujourd'hui » sont réalisées ; les phases à droite correspondent au travail restant : l'évaluation sur le corpus complet et la finalisation du rapport.

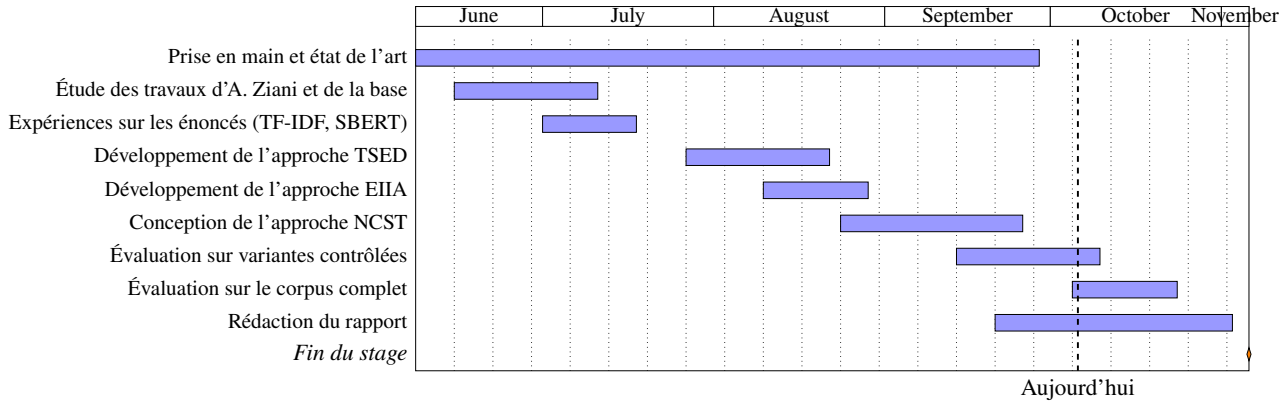


Figure 2.2 – Diagramme de Gantt du stage

2.3 Méthodologie

Ce stage étant un travail de recherche, la démarche n'était pas entièrement fixée à l'avance : elle s'est construite par étapes, en ajustant la direction en fonction des résultats obtenus. Elle suit les trois attendus du sujet (état de l'art, méthodes, validation) et peut se résumer en cinq étapes (figure 2.3) :

1. **Étude bibliographique** : lire les travaux existants sur la similarité entre exercices, algorithmes et codes, afin de comprendre les approches possibles et leurs limites.
2. **Exploration** : reprendre la base d'exercices et les travaux d'A. Ziani, puis tester rapidement plusieurs pistes, comme la comparaison des énoncés par TF-IDF et SBERT, afin d'identifier celles qui donnent un signal utile. Les ambiguïtés et la variabilité des énoncés ont conduit à retenir le code des corrections comme source principale d'information.
3. **Conception et implémentation** : développer les trois approches retenues, EIIA, TSED et NCST.
4. **Validation et calibration** : vérifier le comportement de chaque approche, d'abord sur des variantes contrôlées, c'est-à-dire des programmes modifiés volontairement d'une seule façon, puis sur un corpus plus large.
5. **Analyse et rédaction** : interpréter les résultats, identifier les limites et rédiger le rapport.

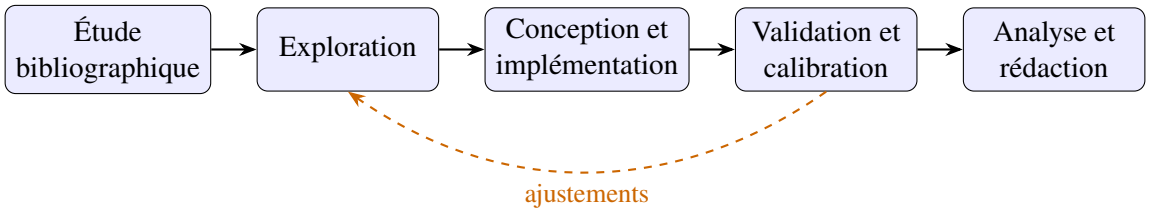


Figure 2.3 – Démarche de travail suivie pendant le stage

Cette démarche est itérative : les résultats de la validation ont souvent conduit à revenir sur une approche pour la corriger ou l'améliorer.

Chapitre 3

État de l'art et positionnement

Ce chapitre dresse un panorama des recherches actuelles concernant l'analyse de code source, particulièrement dans un contexte éducatif. Avant de détailler l'architecture de notre propre pipeline d'analyse, il est essentiel de comprendre comment le concept même de similarité est défini et mesuré par la communauté scientifique.

Dans un premier temps, nous définirons formellement la notion de similarité appliquée aux exercices, en distinguant les différentes manières de l'appréhender en informatique. Nous explorerons ensuite les domaines d'application et les principales approches classiques utilisées dans les Environnements Informatiques d'Apprentissage Humain (EIAH). Nous détaillerons par la suite les outils techniques et les méthodes spécifiques mobilisés par ces travaux, tels que les Arbres de Syntaxe Abstraite (AST) ou les métriques de distance. Enfin, une discussion critique mettra en évidence les limites de ces méthodes existantes. Cette analyse nous permettra de justifier notre positionnement scientifique et de motiver les choix techniques qui constituent le cœur de notre proposition, présentée dans le chapitre suivant.

3.1 Qu'est-ce que la similarité ?

En informatique, et plus particulièrement dans l'évaluation d'exercices de programmation, la similarité désigne la mesure du degré de ressemblance entre deux fragments de programme. Toutefois, dans la littérature scientifique, cette notion n'est pas un concept univoque ni une propriété absolue. Comme le souligne Pelánek [16] dans sa revue systématique, il n'existe pas de mesure universelle : la définition même de ce qui constitue la « proximité » est une construction conceptuelle qui varie radicalement selon le problème à résoudre et les données disponibles.

Concrètement, les chercheurs considèrent que deux codes sont similaires selon trois grands paradigmes :

— **La similarité par l'origine (l'intention de copie) :** Dans les travaux liés à la détection de plagiat, deux codes sont considérés comme similaires s'ils partagent une origine commune trahissant une copie. La similarité se mesure à la quantité de particularités d'écriture communes (erreurs identiques, style de formatage, choix de variables atypiques), et ce, même si l'un des programmes a subi des transformations de surface (offuscation, renommage) pour masquer cette filiation. Cette conception est notamment illustrée par JPlag, qui compare des programmes d'étudiants afin d'identifier des cas de plagiat [17]. Les travaux fondés sur les arbres syntaxiques abstraits étendent également cette idée en recherchant des clones structurellement proches dans le code [2].

— **La similarité par l'équivalence (l'approche formelle) :** D'un point de vue plus théorique, certains travaux définissent la similarité comme une stricte équivalence comportementale. Deux programmes sont jugés similaires s'ils calculent la même fonction mathématique (ils produisent les mêmes sorties pour les mêmes entrées). Cette approche cherche à abstraire totalement le code pour le traduire en modèles formels. La vérification formelle de l'équivalence de programmes est notamment étudiée par [12]. Dans le contexte de l'évaluation d'exercices, des vérificateurs formels peuvent comparer les soumissions des étudiants aux solutions de référence [13]. Si l'équivalence est prouvée, les codes sont déclarés similaires, quand bien même leurs syntaxes n'auraient rien en commun.

— **La similarité par les caractéristiques (l'approche structurelle) :** Enfin, une vision pragmatique considère que la similarité repose sur le partage de caractéristiques mesurables de résolution. Deux codes sont évalués comme proches s'ils affichent des métriques logicielles comparables ou s'ils s'appuient sur un squelette algorithmique analogue (même enchaînement et imbrication des structures de contrôle). Ces caractéristiques peuvent être extraites à partir d'arbres syntaxiques abstraits et de distances d'édition [2, 25, 20], de graphes de flot de contrôle [22] ou de graphes de dépendances [24, 18]. Les approches combinant des métriques et des graphes normalisés cherchent en outre à détecter des programmes proches malgré les variations de syntaxe et de style [23, 10].

Dans le domaine des EIAH, l'objectif n'est ni de traquer le plagiat, ni de prouver une équivalence mathématique stricte. La similarité vise à identifier des activités susceptibles d'être rapprochées pour la recherche, la recommandation ou la personnalisation des parcours d'apprentissage [9, 21]. Elle ne se réduit donc pas à une simple ressemblance textuelle entre les énoncés : deux exercices peuvent être formulés avec des mots très différents tout en mobilisant une démarche algorithmique identique, et inversement.

Dans le contexte de ce rapport, nous définissons la similarité entre deux exercices comme le **degré de proximité entre les caractéristiques pédagogiques et structurelles de leurs solutions de référence**. Ces caractéristiques concernent l'organisation des instructions, les structures de contrôle utilisées, les notions algorithmiques mobilisées et la complexité du traitement. Pour les capturer, l'analyse s'appuie sur des représentations telles que les arbres syntaxiques abstraits (AST) [2], les distances d'édition entre arbres [25] et une représentation issue de Wang *et al.* [23].

On peut formaliser cette idée par une fonction de similarité S associant à deux exercices e_i et e_j un score continu :

$$S(e_i, e_j) \in [0, 1]$$

Une valeur proche de 1 indique une forte proximité algorithmique (stratégies de résolution analogues), tandis qu'une valeur proche de 0 signale des approches fondamentalement distinctes.

Ainsi, la similarité constitue dans ce travail un **indicateur multidimensionnel**. Estimée automatiquement à partir des propriétés intrinsèques du code de l'enseignant, cette valeur ne constitue pas à elle seule un jugement pédagogique définitif. Une forte similarité structurelle permet de filtrer des paires candidates et de signaler que deux exercices sont potentiellement substituables, mais cette substituabilité doit au final être confirmée par une analyse humaine tenant compte des notions visées et des objectifs d'apprentissage.

3.2 Domaine et approches des travaux sur les similarités dans les EIAH

La mesure de similarité entre exercices constitue un problème important dans les Environnements Informatiques pour l'Apprentissage Humain (EIAH). Elle intervient dans plusieurs tâches : l'évaluation automatique des productions, la détection de plagiat ou de clones, la recherche de ressources, la recommandation pédagogique, l'aide à l'apprentissage de la programmation et la personnalisation des parcours d'apprentissage. Ces tâches ont **un objectif commun – « rapprocher des productions ou des activités »** – mais ne donnent pas le même **sens** à la notion de similarité. Une méthode adaptée à la détection d'une copie n'est donc pas nécessairement adaptée à la recommandation d'un exercice d'entraînement.

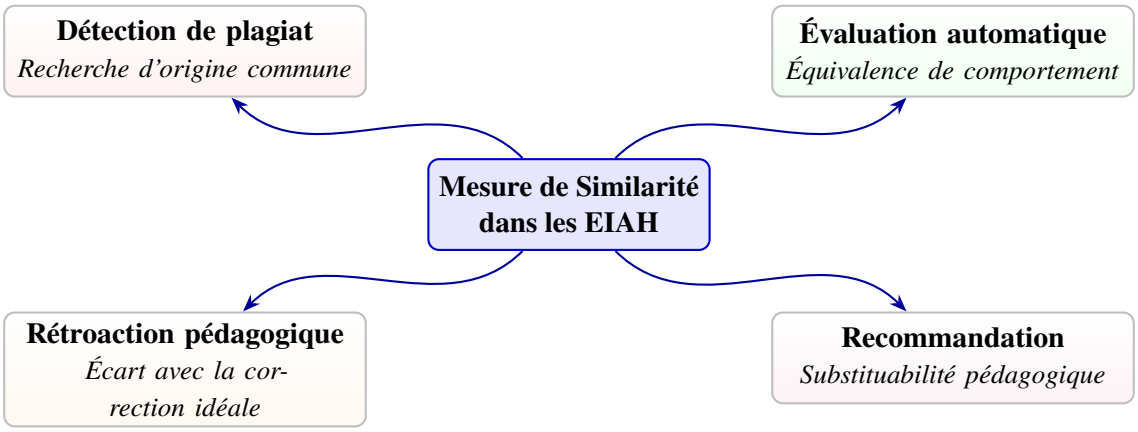


Figure 3.1 – Les quatre grandes finalités de la mesure de similarité dans les EIAH (orientations divergentes à partir d'un même besoin technique).

Dans les premières applications, la comparaison de programmes a surtout été utilisée pour repérer des ressemblances suspectes entre les travaux d'étudiants. Les systèmes de détection de plagiat cherchent alors des indices d'une origine commune : séquences d'instructions, choix de structures, erreurs identiques ou organisation inhabituelle du programme. Des outils comme JPlag comparent les structures des programmes tout en tolérant certaines transformations superficielles, par exemple le renommage des variables ou la modification de la mise en forme [17].

La similarité est également mobilisée pour l'évaluation automatique des réponses. Une solution soumise peut être comparée à une correction de référence ou à un ensemble de solutions déjà validées. Les approches les plus simples examinent la sortie du programme sur plusieurs jeux de tests, tandis que d'autres analysent sa structure, ses dépendances ou ses propriétés sémantiques. La vérification formelle peut aller jusqu'à établir une équivalence comportementale entre deux programmes [13].

Un autre domaine d'application est la recommandation de ressources. Dans une plateforme d'exercices, un enseignant peut souhaiter retrouver rapidement des activités proches d'un exercice existant, soit pour proposer une variante, soit pour construire une évaluation équilibrée. De la même manière, un étudiant peut recevoir un exercice supplémentaire qui reprend une notion déjà

travaillée, sans reproduire exactement le même énoncé. Les travaux sur la recommandation dans les environnements éducatifs cherchent notamment à exploiter les caractéristiques des ressources et les traces d'apprentissage afin d'adapter les propositions au profil de l'apprenant [9, 21].

La comparaison de code intervient également dans la production de rétroactions destinées aux apprenants. Dans l'environnement AlgoPython, Jolivet *et al.* [8] étudient la modélisation et la décision de rétroactions épistémiques adaptées aux erreurs observées et au profil de l'apprenant. Leur approche compare notamment l'arbre syntaxique abstrait d'une production de l'étudiant avec celui d'une solution de référence afin d'identifier les différences structurelles et de les exploiter pour construire un retour pédagogique. La similarité entre ces deux représentations ne sert donc pas ici à détecter une copie ou à recommander un exercice, mais à caractériser l'écart entre la production et la solution attendue. Ce travail montre que les représentations syntaxiques peuvent être utilisées dans plusieurs fonctions d'un EIAH, notamment pour assister l'apprentissage de la programmation.

Deux sources d'information peuvent alors être distinguées :

— les informations externes au programme, comme le texte de l'énoncé, les notions pédagogiques déclarées, le niveau de difficulté ou les métadonnées de l'exercice ;
— les informations internes à la solution, comme les structures de contrôle, les opérations effectuées, l'organisation des données et la complexité approximative du traitement.

En fonction de la source de données considérée, les travaux recensés mettent en œuvre plusieurs familles d'approches complémentaires. Certaines analysent le texte de l'énoncé, d'autres s'appuient sur les concepts pédagogiques ou les traces d'apprentissage, tandis que les approches centrées sur le code étudient sa syntaxe, ses structures de contrôle, ses dépendances ou son comportement à l'exécution. Les méthodes d'apprentissage automatique, notamment les représentations vectorielles et les approches multimodales, ainsi que les méthodes formelles constituent d'autres familles, respectivement fondées sur l'apprentissage de représentations et sur la preuve de propriétés du programme. Enfin, certaines méthodes réutilisent ces représentations pour analyser l'écart entre une production et une référence et produire une rétroaction pédagogique. Aucune de ces approches n'est fiable à 100 % : chacune observe une dimension particulière de l'exercice. Nous les présentons donc ici dans leurs grandes lignes afin de situer les méthodes étudiées dans la littérature. Les outils et les étapes de calcul propres à chaque famille seront détaillés dans la section suivante, avant que leurs limites ne soient discutées au regard de l'objectif de ce rapport.

3.3 Quels sont les outils et méthodes utilisés par ces approches ?

Selon l'objectif poursuivi, la comparaison peut porter sur le texte de l'énoncé, les concepts pédagogiques associés, les traces d'activité, la forme du programme, ses dépendances ou son comportement à l'exécution. Les outils employés diffèrent donc d'une famille à l'autre :

— **le texte de l'énoncé**, représenté par des vecteurs de mots (TF–IDF, FastText) ou de phrases (Sentence-BERT), puis comparé par la similarité cosinus [21, 19] ;
— **les concepts pédagogiques et les traces d'apprentissage**, qui décrivent l'exercice par les notions qu'il mobilise ou par l'activité des apprenants [7, 9] ;
— **les tests et les métriques de programme**, qui exécutent les solutions ou comptent leurs constructions (lignes, boucles, complexité cyclomatique) [11] ;

— **les arbres syntaxiques abstraits (AST)**, comparés par une distance d'édition entre arbres, utilisés aussi bien pour détecter des clones que pour produire une rétroaction [2, 25, 20, 8] ;
— **les graphes de flot et de dépendances** (CFG, DDG, PDG, CPG), qui décrivent les chemins d'exécution et les relations entre variables [22, 24, 15] ;
— **les modèles appris** (code2vec, CodeBERT, GraphCodeBERT), qui projettent le code dans un espace vectoriel [1, 4, 5] ;
— **la vérification formelle**, qui cherche à prouver l'équivalence de deux programmes [12, 13].

Le fonctionnement détaillé de chacune de ces familles est présenté dans le rapport complémentaire (chapitre A).

Synthèse des méthodes mobilisées. Les approches recensées peuvent être regroupées selon la source d'information qu'elles exploitent, comme le résume la figure 3.2. Le schéma ne représente pas une hiérarchie de méthodes ni un classement de leur coût : chaque famille observe une dimension différente des exercices. Dans ce travail, les tests unitaires n'ont pas constitué une méthode de similarité étudiée en profondeur. Ils ont seulement été envisagés lors d'une première exploration comme un moyen de vérifier le comportement des programmes. Les résultats présentés ensuite reposent sur l'analyse statique des corrections de référence ; aucune conclusion bibliographique ou expérimentale spécifique sur le coût des tests unitaires n'est donc tirée ici.

Ces familles sont complémentaires. Les méthodes textuelles et conceptuelles décrivent l'intention déclarée de l'exercice ; les traces décrivent son usage ; l'AST décrit sa construction syntaxique ; les graphes décrivent ses flux et ses dépendances ; les modèles vectoriels apprennent des représentations distribuées ; enfin, la vérification formelle cherche à établir une propriété logique ou comportementale. Les outils associés vont de la similarité cosinus et de la distance d'édition aux algorithmes de comparaison de graphes, aux modèles Transformers et aux solveurs formels.

Dans le cadre de ce rapport, aucune de ces représentations ne suffit à elle seule à caractériser la substituabilité pédagogique. Nous retenons donc une analyse centrée sur les corrections de référence, qui combine une mesure globale de caractéristiques, une comparaison d'AST normalisés et une représentation des structures de contrôle. Ce choix conserve l'interprétabilité des méthodes syntaxiques et structurelles tout en permettant de comparer des stratégies de résolution. Les limites de ces outils, ainsi que les raisons pour lesquelles certaines méthodes ne sont pas retenues dans notre pipeline, sont examinées dans la section suivante.

3.4 Discussion sur les limites des méthodes

Les méthodes présentées dans les sections précédentes permettent de comparer des exercices selon des dimensions différentes, mais aucune ne mesure à elle seule la substituabilité pédagogique. Cette limite tient d'abord à la nature de la notion étudiée. La substituabilité ne dépend pas uniquement de la forme du code : elle suppose également que deux exercices mobilisent des connaissances proches, visent des apprentissages comparables et présentent un niveau de difficulté compatible. Or ces informations ne sont pas toujours explicitement décrites dans les énoncés ou dans les corrections de référence.

Les approches fondées sur le texte constituent un premier moyen de retrouver des exercices traitant de thèmes voisins. Elles sont relativement simples à mettre en œuvre et peuvent être appliquées à un corpus important. Leur principale faiblesse est cependant leur dépendance à la formulation. Deux énoncés qui mobilisent la même démarche algorithmique peuvent employer un vocabulaire très différent. À l'inverse, deux énoncés partageant de nombreux termes peuvent demander des solutions

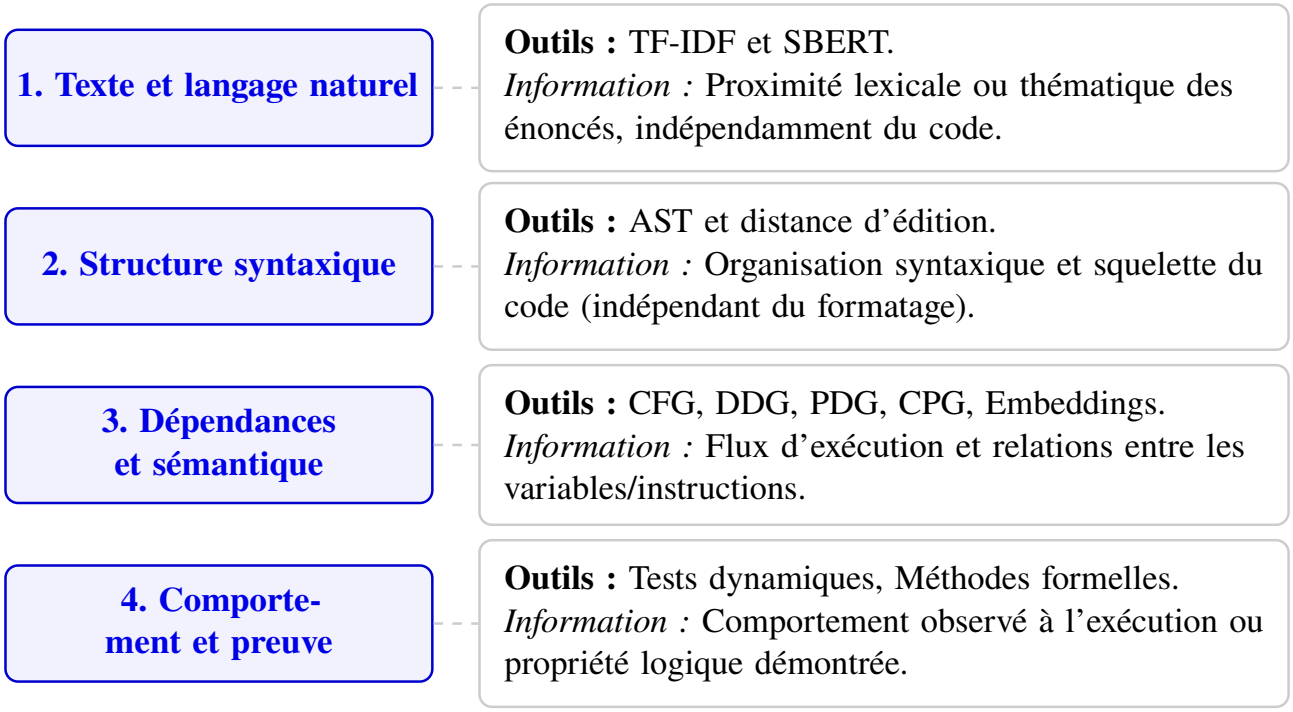


Figure 3.2 – Familles de représentations mobilisées pour comparer des exercices de programmation

très différentes. Les représentations par TF–IDF restent sensibles au partage lexical, tandis que les embeddings contextuels dépendent du modèle utilisé et de ses données d'entraînement. Elles décrivent donc davantage la proximité thématique ou linguistique que la structure effective de la résolution.

Les annotations pédagogiques et les traces d'apprentissage pourraient apporter une information plus directement liée aux objectifs de formation. Des notions, des compétences, des niveaux de difficulté, le temps de résolution ou le taux de réussite permettraient par exemple de compléter l'analyse du code. Ces données ne sont toutefois pas disponibles dans le contexte étudié. Le corpus ne contient pas d'annotation de substituabilité entre les exercices et aucune trace d'activité d'apprenants n'est exploitable. Cette situation de *cold start* empêche notamment d'entraîner de manière fiable un modèle supervisé ou de vérifier directement qu'un score automatique correspond à un jugement pédagogique.

Les méthodes fondées sur l'exécution fournissent une information utile sur le comportement d'un programme. Elles peuvent vérifier que deux solutions produisent les mêmes résultats sur un ensemble d'entrées et mesurer certains aspects de leur performance. Elles ne suffisent cependant pas à caractériser la stratégie algorithmique. Deux programmes peuvent obtenir les mêmes sorties sur les tests disponibles tout en reposant sur des structures très différentes. De plus, une égalité de comportement ne signifie pas nécessairement que les deux exercices poursuivent le même objectif pédagogique. Les tests ne couvrent par ailleurs qu'une partie des comportements possibles et peuvent manquer des cas limites.

Les AST et les distances d'édition offrent une représentation plus stable que le texte brut, car ils réduisent l'influence de la mise en forme et de certains détails lexicaux. Ils restent néanmoins sensibles aux choix d'implémentation. Un changement de boucle, l'extraction d'une fonction ou l'introduction d'une variable intermédiaire peut produire une distance importante alors que la démarche générale reste proche. À l'inverse, deux arbres peuvent présenter une organisation similaire tout en correspondant à des traitements différents. La normalisation atténue ces effets, mais chaque règle de normalisation introduit un choix qui peut supprimer une information pertinente ou rapprocher à tort des programmes distincts.

Les graphes de flot et de dépendances décrivent plus explicitement le fonctionnement du programme. Les CFG, DDG, PDG et CPG peuvent notamment représenter les chemins d'exécution, les dépendances entre variables et les relations entre instructions. Cette richesse permet de dépasser certaines limites des AST, mais elle augmente le coût d'extraction et de comparaison. La construction de ces graphes peut aussi dépendre du langage, de la qualité de l'analyse statique et de la capacité des outils à traiter les appels de fonctions, les bibliothèques ou les effets indirects. Une représentation plus riche n'est donc pas automatiquement plus pertinente pour une comparaison pédagogique.

Les modèles appris et les représentations vectorielles peuvent capturer des relations difficiles à définir manuellement et s'adapter à de grands corpus. Leur interprétation est toutefois moins directe : lorsqu'une paire obtient un score élevé, il est difficile d'identifier précisément les caractéristiques responsables de cette proximité. Leur comportement dépend également du corpus d'entraînement, du langage de programmation et du domaine représenté. Dans le présent travail, l'absence de paires annotées empêche en outre d'apprendre une fonction spécifiquement orientée vers la substituabilité.

Enfin, les méthodes formelles apportent des garanties fortes lorsqu'il est possible de prouver une propriété du programme. Leur coût de mise en œuvre et leurs conditions d'application limitent cependant leur utilisation sur un corpus hétérogène d'exercices. Surtout, l'équivalence comportementale répond à une question différente de celle étudiée ici : deux exercices peuvent demander des résultats différents tout en sollicitant une démarche algorithmique comparable.

Ces limites conduisent à privilégier une approche complémentaire plutôt qu'un score unique présenté comme une vérité pédagogique. Dans ce rapport, les corrections de référence constituent la source principale d'information, car elles permettent d'observer directement les choix structurels de résolution. Les approches retenues combinent une mesure globale de caractéristiques, une comparaison d'AST normalisés et une représentation des structures de contrôle. Ce choix vise un compromis entre robustesse aux variations de code, interprétabilité des résultats et coût de calcul. Les scores obtenus doivent être compris comme des indicateurs destinés à filtrer des paires candidates. Une validation par un enseignant reste nécessaire pour vérifier la proximité des notions visées, de la difficulté et des objectifs d'apprentissage.

Afin d'illustrer la nécessité de croiser les approches, la figure 3.3 résume « l'angle mort » inhérent à chaque grande famille méthodologique vis-à-vis de la notion de substituabilité.

3.5 Pistes explorées et écartées

La définition du pipeline retenu est le résultat de plusieurs explorations préliminaires. L'objectif de ces essais n'était pas seulement d'obtenir le meilleur score possible, mais aussi d'identifier des représentations compatibles avec les données disponibles, suffisamment interprétables et réutilisables sur un corpus d'exercices potentiellement hétérogène. Plusieurs pistes ont ainsi été étudiées sans être retenues comme méthodes principales.

Le tableau 3.1 résume ces pistes et la raison pour laquelle chacune a été écartée. Leur description détaillée, avec les essais réalisés, figure dans le rapport complémentaire (chapitre B).

Ces essais ont conduit à étudier d'abord chaque mesure séparément, avant de les comparer et de les combiner explicitement. Le pipeline présenté au chapitre suivant repose ainsi sur trois représentations complémentaires des corrections : des caractéristiques globales du code, des arbres syntaxiques normalisés et une représentation centrée sur l'organisation des structures de contrôle.

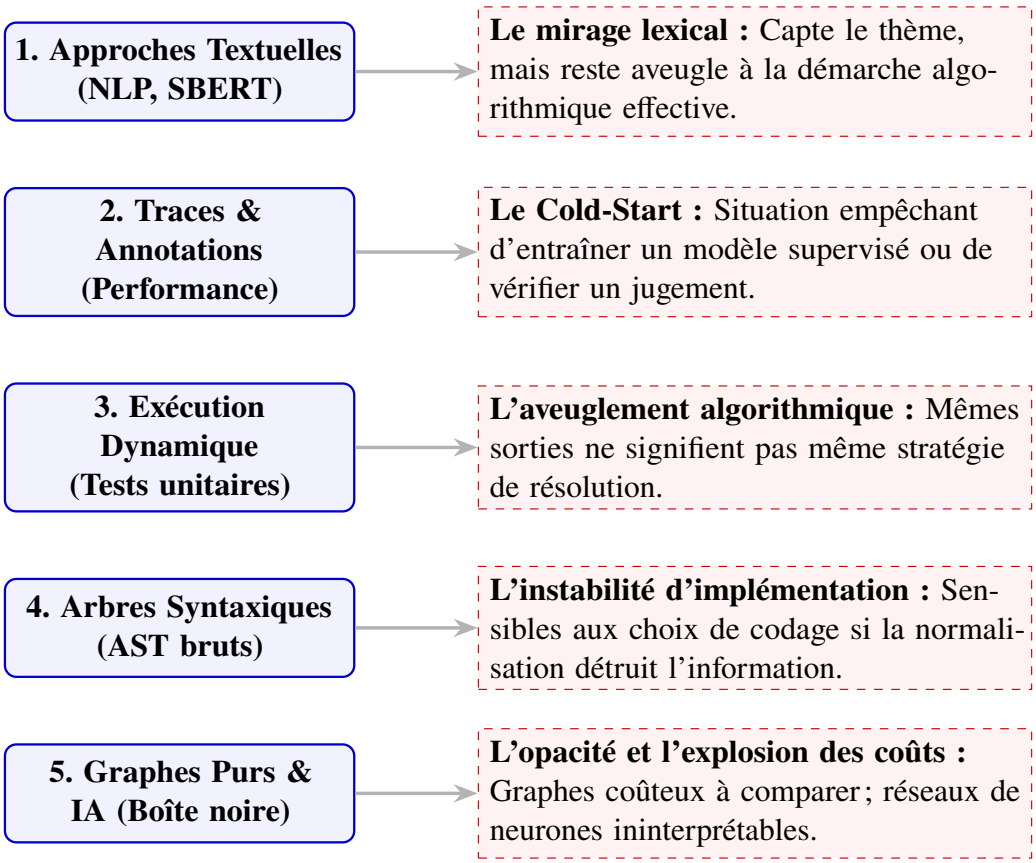


Figure 3.3 – Cartographie des limites : pourquoi chaque approche individuelle échoue à caractériser seule la substituabilité pédagogique.

Piste	Raison de l'abandon
Étiquetage par mots-clés à l'aide de LLM [26]	Étiquettes non reproductibles d'une exécution à l'autre ; vocabulaire fermé qui ignore les notions non prévues.
Motifs syntaxiques extraits avec Tree-sitter	Extraction reproductible, mais limitée aux motifs anticipés dans les règles.
Comparaison des énoncés (TF–IDF, SBERT)	Mesure le thème, pas la démarche : deux énoncés sur des dés obtiennent 0,65 et 0,90 alors que leurs solutions diffèrent.
Séquences de jetons (JPlag) [17]	Cherche des passages recopiés et ignore l'imbrication ; seule l'idée de normalisation a été conservée.
Graphes de dépendances (CFG, PDG, CPG, CSSG)	Comparaison NP-difficile, inapplicable à un grand corpus.
Modèles appris (CodeBERT, GraphCodeBERT)	Optimisés pour la similarité fonctionnelle ; un réentraînement exigerait des paires annotées indisponibles.
Apprentissage contrastif [14]	Pas de paires négatives fiables sans annotation ; l'idée de variantes a été reprise pour l'évaluation (chapitre 5).
Fusion immédiate des scores	Masquerait la contribution de chaque mesure ; poids impossibles à fixer sans vérité terrain.

Table 3.1 – Pistes explorées et raisons de leur abandon.

Chapitre 4

Ma proposition

Ce chapitre présente le pipeline multi-niveaux développé pour évaluer la substituabilité algorithmique entre des exercices de programmation. L'objectif n'est pas de réduire la similarité à une seule propriété du code, mais de comparer les corrections selon trois niveaux d'abstraction complémentaires : la structure globale, la topologie syntaxique détaillée et l'organisation structurelle abstraite du programme. Une interface Streamlit complète le pipeline : elle permet de lancer les comparaisons, de consulter les trois scores et d'inspecter les arbres produits à chaque étape.

4.1 Architecture globale du pipeline

Pour chaque paire de programmes (A, B), le pipeline construit plusieurs représentations du code et calcule trois mesures distinctes :

1. l'**EIIA**, qui compare des profils de complexité globale extraits de l'AST normalisé ;
2. le **TSED**, distance d'édition calculée sur des AST normalisés afin d'évaluer finement la topologie syntaxique ;
3. le **NCST**, qui compare, par un appariement ordonné, les nœuds abstraits d'un arbre structurel normalisé.

Ces trois mesures sont calculées séparément pour chaque paire. Le pipeline produit donc trois scores indépendants, chacun décrivant une dimension particulière de la ressemblance entre les programmes. Cette organisation multi-vues permet notamment de repérer les cas où deux corrections ont une complexité globale proche, mais des organisations structurelles différentes. Elle est cohérente avec les travaux qui distinguent la similarité de contenu, de structure et de finalité pédagogique [16, 6].

La chaîne de traitement est résumée par la figure 4.1. Le code source est d'abord analysé par Tree-sitter, qui produit un arbre syntaxique concret (CST). Cet arbre alimente deux branches : la première le convertit en AST simplifié puis le normalise, ce qui sert de base commune à l'EIIA et au TSED ; la seconde construit directement le NCST brut à partir du CST, le normalise puis apparie les nœuds abstraits. Les trois scores sont exportés dans des colonnes séparées. L'interface Streamlit peut ensuite calculer, à titre facultatif, un score pondéré dont les poids sont choisis par l'utilisateur.

Le tableau 4.1 résume ce que chaque approche conserve, ignore et mesure.

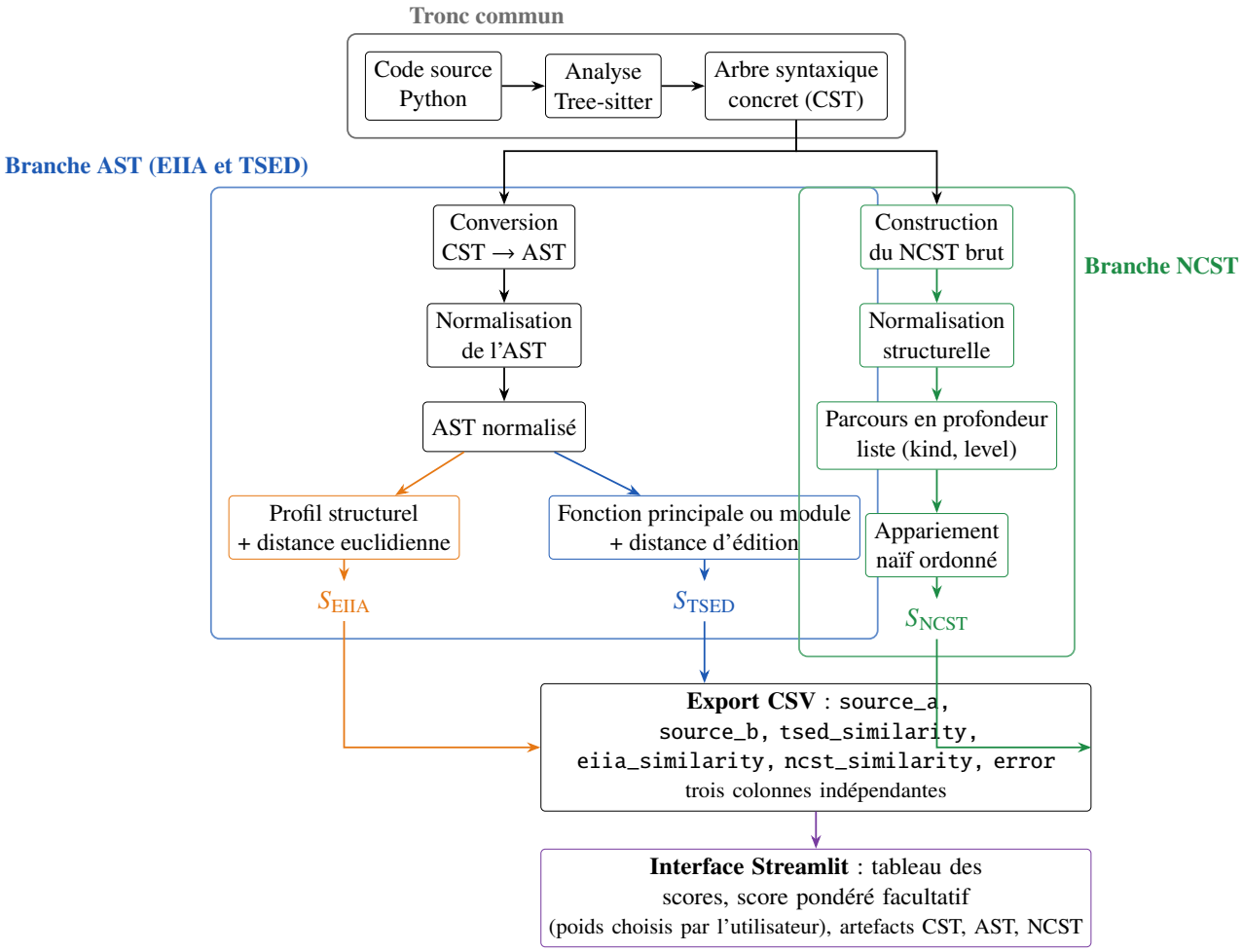


Figure 4.1 – Architecture globale du pipeline d'évaluation de la similarité multi-niveaux. L'EIIA et le TSED partagent l'AST normalisé ; le NCST est construit directement à partir du CST. L'interface Streamlit exploite le CSV produit par le pipeline.

	EIIA	TSED	NCST
Représentation	AST normalisé, résumé en vecteur	AST normalisé complet	NCST normalisé, linéarisé en liste (*kind*, *level*)
Information conservée	Effectifs de nœuds et profondeur	Labels AST, opérateurs, ordre et hiérarchie	Catégories structurelles, niveau d'imbrication, ordre du parcours
Information ignorée	Position et agencement des nœuds	Noms et valeurs concrètes	Noms, valeurs, expressions, opérateurs, paramètres, noms de fonctions
Calcul	Distance euclidienne normalisée	Distance d'édition de Zhang-Shasha	Appariement naïf ordonné, indice de Sørensen-Dice
Question posée	Les programmes ont-ils une taille et une complexité comparables ?	Les arbres sont-ils proches nœud à nœud ?	Les programmes partagent-ils les mêmes catégories abstraites aux mêmes niveaux, dans le même ordre ?

Table 4.1 – Synthèse des trois niveaux de représentation du pipeline.

4.2 Évaluation de l'information indépendante de l'arbre (EIIA)

L'EIIA fournit une mesure macroscopique de la proximité entre deux programmes. Elle repose sur un profil structurel extrait en parcourant l'ensemble des nœuds de l'AST normalisé. Pour un programme A, le profil est représenté par le vecteur à neuf composantes :

$$V_A = (n_{nodes}, n_{operators}, n_{operands}, n_{assignments}, n_{selections}, n_{iterations}, d_{max}, n_{calls}, n_{returns}). \quad (4.1)$$

Le tableau 4.2 précise les types de nœuds comptés pour chaque composante.

Composante	Signification	Nœuds comptés
`n_nodes`	taille de l'arbre	tous les nœuds
`n_operators`	opérateurs	`BINARY_OP, COMPARE, BOOLEAN_OP, UNARY_OP`
`n_operands`	opérandes	`IDENTIFIER, NUMBER, STRING, BOOLEAN, NULL`
`n_assignments`	affectations	`ASSIGN, AUG_ASSIGN`
`n_selections`	sélections	`IF, ELIF, ELSE, TERNARY`
`n_iterations`	répétitions	`FOR, WHILE`
`d_max`	profondeur maximale	—
`n_calls`	appels de fonction	`CALL`
`n_returns`	retours	`RETURN`

Table 4.2 – Composantes du profil structurel utilisé par l'EIIA.

La distance euclidienne entre les profils de deux programmes est définie par :

$$d(V_A, V_B) = \|V_A - V_B\|_2. \quad (4.2)$$

Cette distance est normalisée par la norme maximale des deux profils :

$$d_{norm}(A, B) = \frac{d(V_A, V_B)}{\max\left(\|V_A\|_2, \|V_B\|_2, \varepsilon\right)}, \qquad \varepsilon = 10^{-12}. \quad (4.3)$$

Le score EIIA est alors obtenu par :

$$S_{\text{EIIA}}(A, B) = \max\left(0,\ 1 - d_{norm}(A, B)\right). \quad (4.4)$$

La figure 4.2 illustre cette chaîne de calcul. Aucune pondération explicite n'est appliquée aux composantes. Cette décision ne signifie toutefois pas qu'elles ont la même influence : les composantes conservant leurs échelles brutes, les effectifs de nœuds peuvent dominer les composantes moins fréquentes, comme la profondeur ou le nombre de retours. Cette limite est prise en compte dans l'interprétation de l'EIIA.

Cette approche s'inscrit dans la famille des métriques statiques qui décrivent la taille et la complexité d'un programme sans nécessiter l'alignement de deux arbres ou graphes [22]. Des travaux consacrés à l'évaluation de la difficulté et de l'effort de programmation montrent également que des métriques telles que la taille du code et la complexité cyclomatique peuvent être corrélées à un indice d'effort interne, l'*Index of Internal Effort* (IIE) [3]. Cette observation soutient l'utilisation de profils quantitatifs comme indicateurs de complexité, sans établir pour autant une équivalence entre l'EIIA et l'IIE. L'EIIA constitue ainsi un indicateur de la proximité globale d'architecture et de complexité, sans prétendre décrire à lui seul la stratégie algorithmique détaillée : deux programmes ayant les mêmes effectifs de nœuds obtiennent un score maximal même si ces nœuds sont agencés différemment.

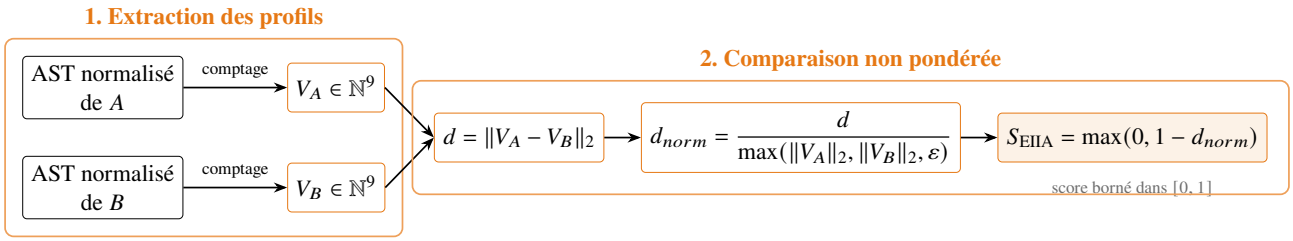


Figure 4.2 – Chaîne de calcul de l'EIIA : extraction de deux profils structurels, puis distance euclidienne normalisée.

4.3 AST normalisé et distance TSED

La deuxième approche évalue la structure grammaticale détaillée du code. Le code Python est d'abord analysé avec Tree-sitter, puis converti en un AST simplifié dont les nœuds portent des types unifiés (`FUNCTION`, `ASSIGN`, `IF`, `FOR`, `BINARY_OP`, `CALL`, `IDENTIFIER`, etc.). Une phase de normalisation réduit ensuite l'influence des choix de style et des détails formels qui ne modifient pas la logique de résolution.

4.3.1 Conversion du CST en AST simplifié

La conversion associe à chaque type de nœud Tree-sitter un type de l'AST interne au moyen d'une table de correspondance. Les mots-clés et la ponctuation sont abandonnés, tandis que les opérateurs sont recopiés comme valeur du nœud (`BINARY_OP(+)`, `COMPARE(==)`). Les constructions qui traduisent un choix de résolution, comme une compréhension de liste, reçoivent un type propre afin que les mesures puissent les distinguer d'une boucle. Un nœud dont le type est absent de la table est supprimé et remplacé par ses enfants, dans le même ordre : l'AST ne contient donc aucune étiquette artificielle du type `UNKNOWN`. Les règles détaillées et leurs justifications sont données dans le rapport complémentaire (section C.1).

4.3.2 Normalisation de l'AST

La normalisation transforme les deux arbres avant le calcul de la distance d'édition. Elle comprend les opérations suivantes :

— l'anonymisation canonique des paramètres, des variables locales et des fonctions du fichier, remplacés dans l'ordre d'apparition par `PARAM_1`, `VAR_1` et `FUNC_1` ;

— l'abstraction des littéraux, qui remplace les valeurs concrètes par les catégories `NUM`, `STR`, `BOOL` et `NULL` ;

— l'*inlining* des fonctions auxiliaires simples. L'appel est remplacé par l'expression renvoyée par la fonction, dans laquelle chaque paramètre est remplacé par l'argument correspondant. Par exemple, avec `def est_pair(n): return n % 2 == 0`, l'appel `if est_pair(x):` devient `if x % 2 == 0:`. Le corps de la fonction n'est donc pas recopié à l'endroit de l'appel : seule son expression de retour est substituée. Cette substitution n'est appliquée qu'aux fonctions définies dans le même fichier et dont le corps se réduit à un unique `return` portant une expression, sans affectation, boucle, condition, appel, exception ni récursion. Les fonctions prédéfinies du langage, comme `sum` ou `len`, et les fonctions importées ne sont jamais remplacées : leur appel reste un nœud `CALL`.

Les opérateurs, tels que +, % ou ==, ainsi que les structures de contrôle sont conservés, car ils portent une information importante sur la stratégie de résolution. Les affectations augmentées, comme `a += b`, sont conservées telles quelles : elles ne sont pas confondues avec l'affectation explicite `a = a + b`. Les commentaires sont supprimés dès la conversion de l'arbre. La figure 4.3 illustre l'effet combiné de ces règles sur une instruction.

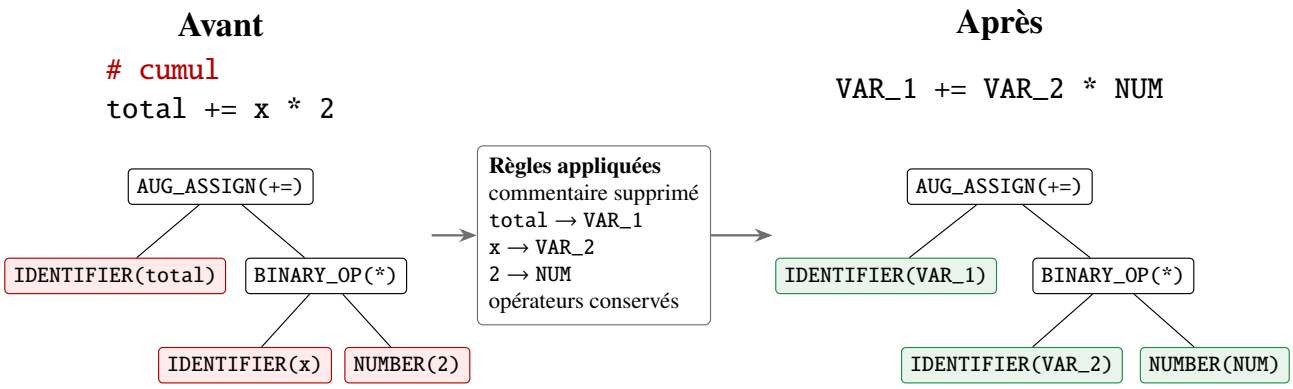


Figure 4.3 – Normalisation d'une instruction. Les nœuds en rouge portent des détails de surface (noms, valeurs) ; après normalisation, ils sont remplacés par des formes canoniques (en vert). Les opérateurs et la forme de l'affectation sont conservés.

4.3.3 Distance d'édition entre arbres

La distance TSED correspond à la distance d'édition entre les deux arbres normalisés, calculée selon l'algorithme de Zhang et Shasha [25]. Chaque nœud est décrit par un label associant son type à sa valeur résiduelle, par exemple `BINARY_OP(+)` ou `IDENTIFIER(VAR_1)`. Les coûts d'insertion, de suppression et de substitution sont unitaires :

$$c_{ins} = c_{del} = 1, \qquad c_{sub}(u, v) = \begin{cases} 0 & \text{si label}(u) = \text{label}(v), \\ 1 & \text{sinon.} \end{cases} \tag{4.5}$$

Si $TED(A, B)$ désigne la distance d'édition et $|A|$ et $|B|$ le nombre de nœuds des deux arbres, le score est défini par :

$$S_{\text{TSED}}(A, B) = 1 - \frac{TED(A, B)}{|A| + |B|}. \tag{4.6}$$

Le dénominateur est la seule borne valable pour toute paire d'arbres : supprimer tous les nœuds de A puis insérer tous ceux de B coûte $|A| + |B|$. La distance peut en revanche dépasser $\max(|A|, |B|)$, car une substitution ne modifie que le label d'un nœud, jamais la forme de l'arbre. Lorsque les formes diffèrent, un nœud mal placé ne peut pas être renommé : il doit être supprimé puis recréé, soit deux opérations. Par exemple, une chaîne de quatre nœuds et un arbre formé d'une racine et de trois feuilles sont à une distance de 6. Ce dénominateur garantit un score compris entre 0 et 1.

La figure 4.4 montre un cas minimal : deux fonctions qui ne diffèrent que par un opérateur. Une seule substitution suffit à transformer un arbre en l'autre.

Pour comparer des arbres de même niveau, le système choisit, pour chaque paire, l'arbre comparé selon la présence de fonctions. Si les deux fichiers contiennent des fonctions, la comparaison porte sur leur fonction principale, celle dont l'AST contient le plus de nœuds (la première en cas d'égalité), afin de comparer en priorité la fonction centrale de la correction. Si au moins l'un des deux fichiers

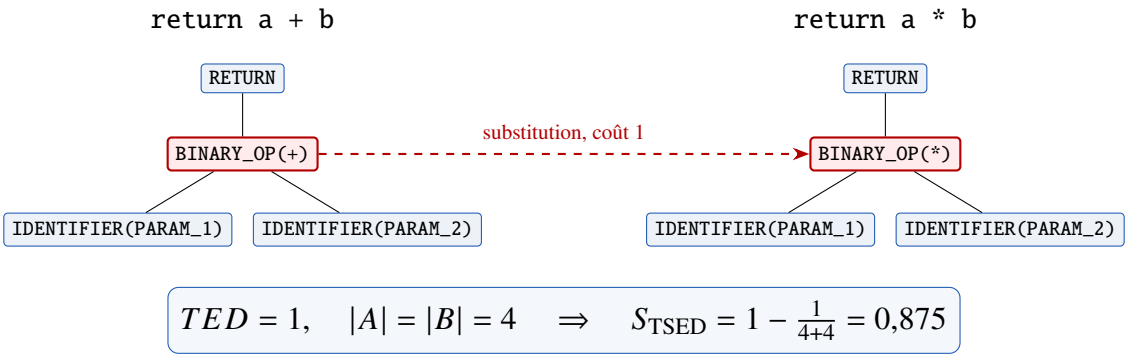


FIGURE 4.4 – Exemple de calcul TSED sur deux AST normalisés de quatre nœuds.

n'en contient aucune, comme un script écrit au niveau du module, la comparaison porte sur le module complet des deux fichiers : une fonction n'est ainsi jamais confrontée à un module entier. L'EIIA est calculé sur le même arbre ; le NCST porte toujours sur le fichier entier.

4.4 Arbre structurel normalisé (NCST)

La troisième approche compare l'organisation structurelle abstraite des programmes. Deux AST peuvent être très différents tout en présentant une organisation proche lorsqu'ils partagent les mêmes catégories de blocs, aux mêmes niveaux d'imbrication et dans le même ordre.

4.4.1 Construction et normalisation du NCST

Le NCST (*Normalized Control Structure Tree*) est construit directement à partir du CST produit par Tree-sitter. Chaque construction structurante (fonction, boucle, sélection, gestion d'exception, etc.) devient un nœud typé relié à ses blocs, et les instructions ordinaires deviennent des nœuds anonymes `STATEMENT`. Une normalisation supprime ensuite tout ce qui dépend de l'écriture : noms de variables, valeurs, expressions, opérateurs, paramètres et noms de fonctions.

Chaque nœud reçoit enfin un *niveau structurel* (*level*) qui n'augmente que lorsque l'on entre dans un bloc, comme le corps d'une boucle ou une branche d'une condition. Une sélection placée dans une itération et une itération placée dans une sélection produisent donc deux arbres différents. Le vocabulaire complet du NCST et le calcul du niveau sont détaillés dans le rapport complémentaire (chapitre D).

4.4.2 Exemple de NCST normalisé et linéarisation

Pour comparer deux NCST, chaque arbre est parcouru en profondeur depuis sa racine. Les nœuds sont placés dans une liste qui conserve l'ordre du parcours, chaque élément étant le couple (*kind*, *level*).

La figure 4.5 présente le NCST normalisé d'une fonction contenant deux instructions, une boucle, une sélection dans cette boucle et une instruction `return`. Les noms de variables, les valeurs et les opérateurs ont disparu : il ne reste que la structure abstraite du programme.

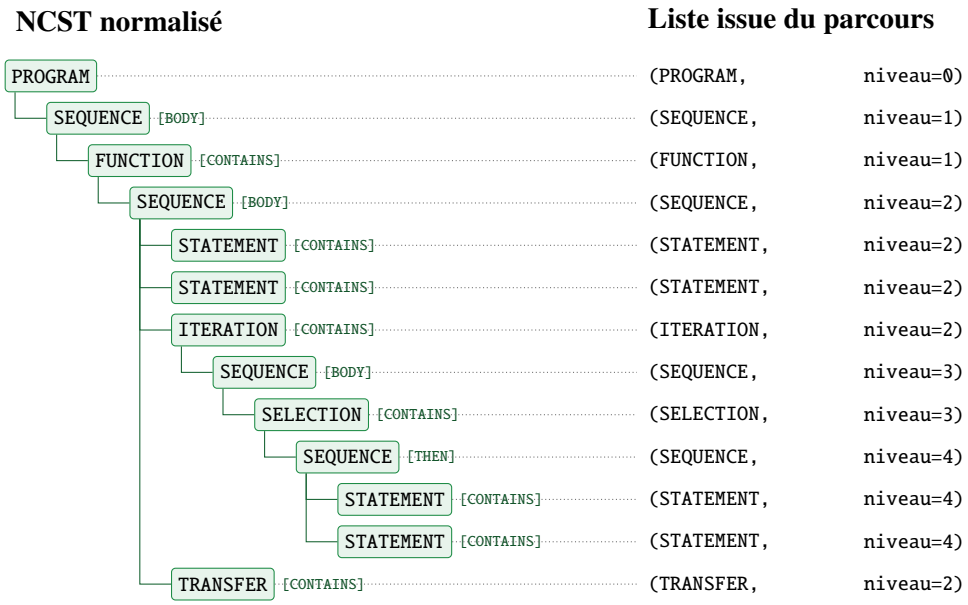


Figure 4.5 – NCST normalisé d'une fonction (gauche), avec entre crochets la relation qui relie chaque nœud à son parent, et liste (*kind*, *level*) obtenue par parcours en profondeur (droite). Le retrait traduit la profondeur dans l'arbre, tandis que le niveau n'augmente que sur les arêtes `BODY` et `THEN`.

4.4.3 Invariance du squelette structurel

La figure 4.6 reprend deux fonctions dont les expressions et les valeurs manipulées sont différentes. Le premier programme recherche un nombre pair, tandis que le second recherche une valeur cible. Malgré cette différence de domaine, les deux solutions parcourent une collection, testent une condition puis retournent immédiatement un résultat en cas de succès, avant d'effectuer un retour par défaut. Après normalisation, les deux NCST produisent la même liste.

Dans cet exemple, le score $S_{\text{NCST}} = 1$ exprime une correspondance parfaite de la structure conservée par la représentation. Il ne signifie pas que les deux fonctions sont équivalentes du point de vue de leur résultat, mais qu'elles partagent une même organisation : itération, test, retour anticipé et retour par défaut.

4.4.4 Algorithme d'appariement naïf ordonné

Choix du petit et du grand arbre. Le nombre de nœuds de chaque NCST normalisé est compté. Le NCST qui contient le moins de nœuds devient le *petit arbre A*, celui qui en contient le plus devient le *grand arbre B*. Le matching est effectué du petit arbre vers le grand arbre, ce qui évite de considérer chaque nœud du grand arbre comme point de départ possible.

Principe. L'algorithme est volontairement naïf : il ne s'agit pas d'un calcul de distance d'édition d'arbres. Pour chaque nœud du petit arbre :

1. le grand arbre est parcouru à partir de la position qui suit le dernier nœud apparié ;
2. le premier nœud compatible est recherché ;
3. deux nœuds sont compatibles s'ils possèdent le même type abstrait (*kind*) et le même niveau structurel (*level*) ;
4. lorsqu'un appariement est trouvé, sa position dans le grand arbre est mémorisée ;

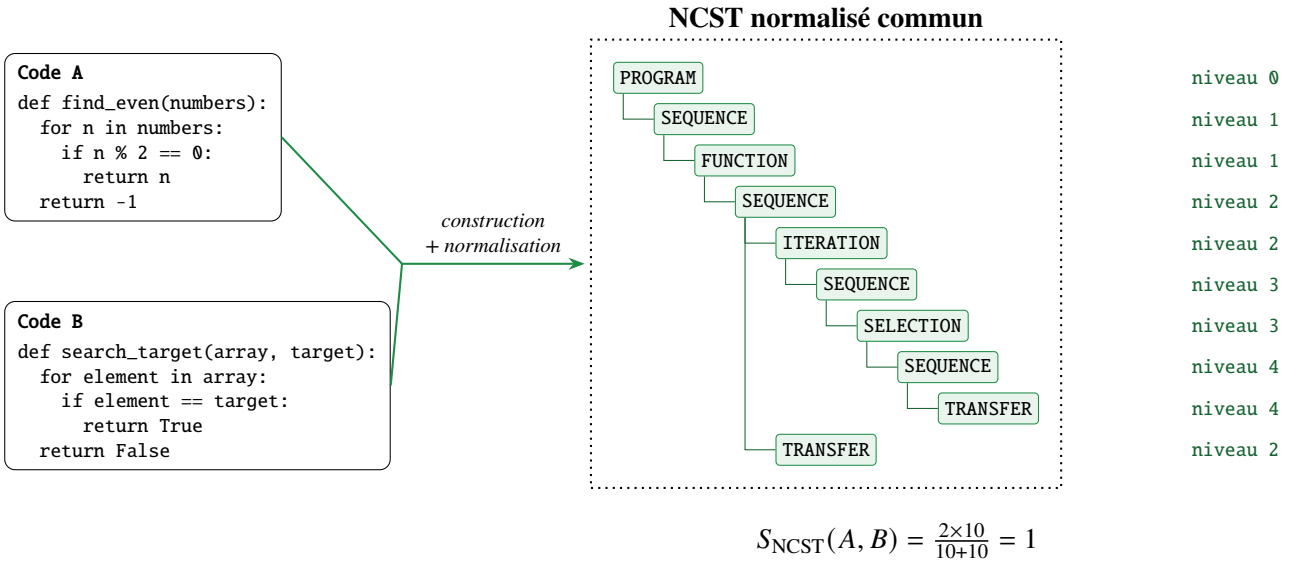


FIGURE 4.6 – Invariance du NCST normalisé sur deux programmes différents. Les codes A et B utilisent des variables, des opérateurs et des valeurs de retour distincts, mais leur normalisation produit la même liste de nœuds abstraits.

5. le nœud suivant du petit arbre ne peut être recherché qu'après cette position.

La condition de compatibilité s'écrit :

$$\text{compatible}(a, b) \iff a.kind = b.kind \ \wedge \ a.level = b.level. \tag{4.7}$$

Exemple d'appariement. La figure 4.7 applique cet algorithme à un petit arbre de cinq nœuds et à un grand arbre de six nœuds. Lors de la recherche de `ITERATION` (niveau 2), le parcours du grand arbre commence à l'indice 2 : le `STATEMENT` de niveau 2 n'est pas compatible et il est dépassé. Ce nœud supplémentaire reste non apparié.

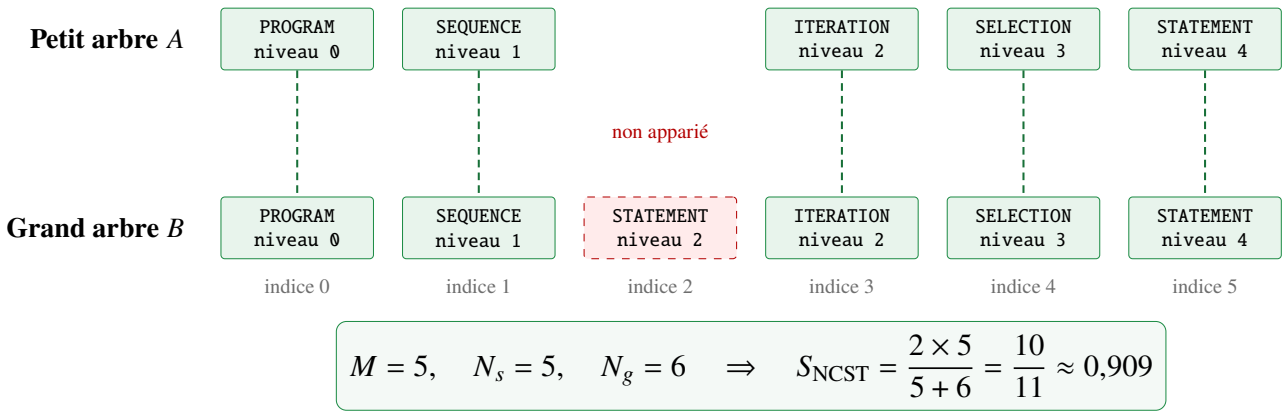


FIGURE 4.7 – Appariement ordonné du petit arbre vers le grand arbre. Les pointillés relient des nœuds de même *kind* et de même *level*, dans un ordre croissant d'indices.

4.4.5 Calcul de la similarité NCST

Soit M le nombre de nœuds appariés, N_s le nombre de nœuds du petit arbre et N_g le nombre de nœuds du grand arbre. La similarité NCST utilise l'indice de Sørensen-Dice :

$$S_{\text{NCST}}(A, B) = \frac{2M}{N_s + N_g}. \tag{4.8}$$

Comme $M \leq N_s \leq N_g$, le score est compris entre 0 et 1 : il vaut 0 lorsqu'aucun nœud n'est apparié et 1 lorsque tous les nœuds des deux arbres sont appariés, ce qui impose $N_s = N_g = M$. Une valeur intermédiaire indique que certains nœuds sont communs, mais que l'un des arbres contient des nœuds supplémentaires ou différents. Par exemple, avec $N_s = 2$, $N_g = 4$ et $M = 2$:

$$S_{\text{NCST}} = \frac{2 \times 2}{2 + 4} = \frac{4}{6} \approx 0{,}6667. \tag{4.9}$$

Même lorsque tous les nœuds du petit arbre trouvent un correspondant, la différence de taille entre les deux arbres limite donc le score.

4.4.6 Positionnement par rapport au TSED et à l'EIIA

Les trois mesures ont des rôles différents. Le TSED compare finement les AST au moyen d'une distance d'édition d'arbres et tient compte des labels et de leur structure détaillée. L'EIIA compare un vecteur de métriques structurelles (nombre de nœuds, d'opérateurs, d'affectations, de sélections, d'itérations, d'appels et de retours). Le NCST, enfin, compare uniquement des nœuds abstraits normalisés à travers leur *kind* et leur *level*, en ignorant les noms, les valeurs et les expressions, mais en conservant l'ordre général du parcours. Le NCST n'est donc pas un *Tree Edit Distance* : il s'agit d'un appariement ordonné et naïf de nœuds abstraits. Le score obtenu est une heuristique de proximité structurelle.

4.5 Exécution et synthèse des mesures

Le pipeline ne fusionne pas les trois mesures. Elles décrivent des dimensions distinctes de la similarité logicielle, et leur combinaison prématurée masquerait la contribution de chaque représentation. Pour chaque paire de programmes, le système exporte donc séparément les trois scores S_{EIIA}, S_{TSED} et S_{NCST}, selon le format présenté dans le tableau 4.3.

source_a	source_b	tsed_similarity	eiia_similarity	ncst_similarity	error
ex_A.py	ex_B.py	S_{TSED}	S_{EIIA}	S_{NCST}	—
ex_A.py	ex_C.py	S_{TSED}	S_{EIIA}	S_{NCST}	—
			...		

Table 4.3 – Structure du fichier CSV exporté par le pipeline : une ligne par paire, une colonne par mesure, et une colonne d'erreur en cas d'échec de l'analyse.

Cette sortie conserve une information interprétable sur la nature de la proximité. Le tableau 4.4 donne quelques profils de lecture typiques : par exemple, deux exercices peuvent obtenir un score EIIA

EIIA	TSED	NCST	Lecture possible
élevé	élevé	élevé	Corrections très proches, à la fois en taille, en syntaxe et en structure.
élevé	faible	élevé	Même organisation structurelle et même complexité, mais écritures détaillées différentes.
faible	faible	élevé	Même organisation appliquée à des expressions de tailles différentes.
élevé	—	faible	Complexité comparable mais organisations structurelles distinctes.

Table 4.4 – Exemples de profils de scores et de leur interprétation.

élevé, signe d'une architecture globale comparable, tout en présentant un score NCST faible, indiquant une organisation structurelle différente.

Les trois colonnes permettent ainsi d'analyser séparément les dimensions de la substituabilité algorithmique avant toute interprétation pédagogique.

4.6 Interface de comparaison Streamlit

Une interface web développée avec Streamlit rend le pipeline utilisable sans passer par la ligne de commande. Elle ne réimplémente pas les mesures : elle prépare un jeu de fichiers, appelle la fonction `run_pipeline` du module `run_poc`, lit le CSV produit, puis affiche les scores et les arbres intermédiaires. La figure 4.8 résume son fonctionnement.

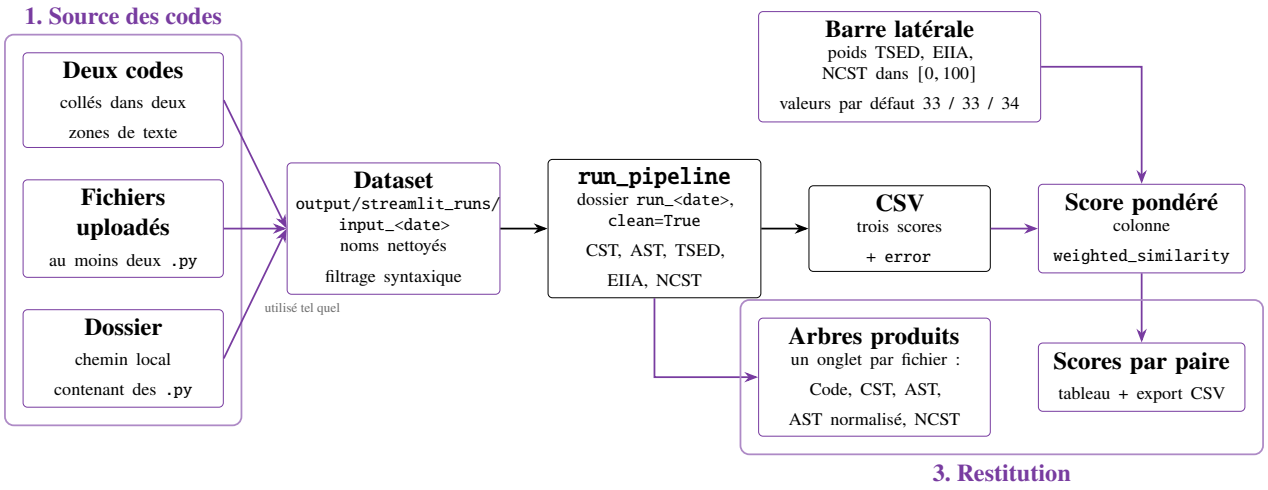


Figure 4.8 – Fonctionnement de l'interface Streamlit : préparation du jeu de fichiers, appel du pipeline, pondération facultative et restitution des scores et des arbres.

L'interface accepte deux codes collés, des fichiers envoyés ou un dossier local. Avant de lancer le calcul, elle écarte les fichiers syntaxiquement invalides, qui recevraient sinon des scores sans rapport avec leur structure. La barre latérale permet de choisir les poids d'un score pondéré facultatif, et chaque fichier dispose d'un onglet affichant son code et ses arbres intermédiaires (CST, AST, AST normalisé, NCST). Lorsqu'un score surprend, l'utilisateur peut ainsi comparer directement les représentations qui l'ont produit. Le fonctionnement détaillé de l'interface est décrit dans le rapport complémentaire (chapitre E).

4.7 Protocole d'évaluation des approches

L'évaluation se déroule en deux phases, résumées par la figure 4.9. Les trois scores sont d'abord calculés sur des variantes contrôlées, obtenues en appliquant une transformation identifiée à une correction de référence. Cette étape permet d'observer séparément la sensibilité de chaque mesure à un renommage, à une modification de structure de contrôle ou à une transformation de l'organisation du code. Les résultats sont ensuite produits sur le corpus complet afin d'étudier la distribution des scores et d'identifier les paires candidates pour une analyse pédagogique. L'interface Streamlit sert de support à cette analyse : elle permet de rejouer une paire et d'inspecter ses arbres intermédiaires.

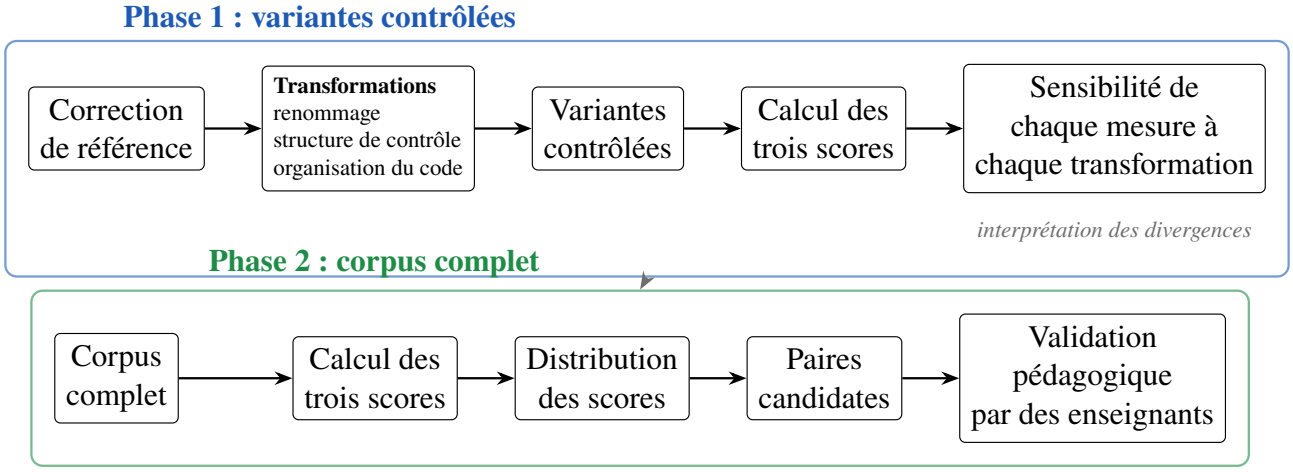


Figure 4.9 – Protocole d'évaluation en deux phases.

Les mesures ne sont pas interprétées comme une vérité terrain automatique. La comparaison porte d'abord sur leur comportement attendu face aux variantes, puis sur leur complémentarité. Les cas où les scores divergent sont conservés comme des situations informatives : ils permettent d'identifier la dimension de la similarité capturée par chaque approche et de préparer une validation ultérieure par des enseignants, seuls à même de juger si deux exercices sont réellement substituables d'un point de vue pédagogique.

Chapitre 5

Résultats expérimentaux

Ce chapitre présente le corpus, les variantes contrôlées et les résultats du pipeline. Les trois mesures sont analysées séparément afin d'identifier la dimension de similarité capturée par l'EIIA, le TSED et le NCST. En l'absence d'annotations pédagogiques exhaustives, les scores sont interprétés comme des indicateurs de comparaison et non comme une vérité terrain.

5.1 Constitution du corpus et variantes contrôlées

Deux jeux de données sont utilisés. Le corpus principal, issu des travaux d'A. Ziani, comprend 906 exercices écrits en Python, chacun associé à un énoncé et à une correction de référence, soit 409 965 paires à comparer ; ses corrections comptent en moyenne 13,39 lignes. Le jeu de variantes contrôlées, construit manuellement, comprend 13 fichiers Python : la correction de référence et ses 12 variantes. Pour chaque correction, le pipeline extrait les représentations nécessaires au calcul des trois scores. Les erreurs de traitement survenues lors de la comparaison d'une paire sont conservées dans la colonne `error` de l'export CSV.

Tree-sitter ne lève pas d'erreur sur un code invalide (rapport complémentaire, section E.2). Un contrôle syntaxique préalable montre que 100 des 906 corrections du corpus complet sont invalides. Dans 96 cas, le code provient de GitHub et reste encadré par les balises Markdown ```` ```python ```` et ```` ``` ```` : il suffit de retirer ces lignes pour obtenir un programme valide, sans modifier le code lui-même. Les quatre autres fichiers ne sont pas des programmes Python exploitables : un code écrit en C, une phrase en anglais placée avant le code, une adresse électronique altérée lors de la récupération de la page web et un littéral numérique invalide.

Les variantes contrôlées sont produites à partir d'une correction de référence choisie aléatoirement dans le corpus. Chaque variante applique une transformation identifiée : renommage des variables, modification de la syntaxe, changement d'une structure de contrôle, ajout ou suppression d'une fonction auxiliaire, ou modification d'une expression, d'un opérateur ou d'un littéral. Cette construction permet de comparer le comportement attendu des mesures avant l'analyse du corpus complet.

À titre d'illustration, la figure 5.1 met en regard la correction de référence `01_base.py` et la variante `9_while_index.py`. Les deux programmes calculent exactement le même résultat, la somme et le nombre des éléments pairs, et suivent la même démarche : parcourir la liste, tester la parité, mettre à jour deux accumulateurs. Seule la manière de parcourir la liste change. La boucle `for` est remplacée par une boucle `while` pilotée par un indice explicite, ce qui introduit trois instructions absentes de la référence : l'initialisation `i = 0`, l'accès `x = liste[i]` et l'incrémentation `i += 1`. Cette variante

Métrique	Valeur
Nombre total de fichiers	906
Nombre total de lignes	12 129
Nombre maximum de lignes	216
Nombre moyen de lignes	13,39
Paires comparées	409 965

Table 5.1 – Statistiques descriptives du corpus de corrections de référence.

Métrique	Valeur
Nombre total de fichiers	13
Nombre total de lignes	133
Nombre maximum de lignes	28
Nombre moyen de lignes	11,08
Paires comparées	78

Table 5.2 – Statistiques descriptives du jeu de variantes contrôlées.

est intéressante car elle conserve la stratégie de résolution tout en modifiant sensiblement l'arbre syntaxique : une mesure de substituabilité devrait donc la juger proche de la référence, alors qu'une mesure trop sensible à la syntaxe risque de la pénaliser.

`01_base.py` (référence)

```python
def stats_pairs(liste):
    total = 0
    nb = 0
    for x in liste:
        if x % 2 == 0:
            total += x
            nb += 1
    return total, nb
```

`9_while_index.py` (variante)

```python
def stats_pairs(liste):
    total = 0
    nb = 0
    i = 0                  # ajout
    while i < len(liste): # for->while
        x = liste[i]       # ajout
        if x % 2 == 0:
            total += x
            nb += 1
        i += 1             # ajout
    return total, nb
```

Figure 5.1 – Correction de référence et variante contrôlée « `while` avec index » : même stratégie, structure de parcours différente.

5.2 Résultats sur les variantes contrôlées

Le jeu de variantes contrôlées est construit autour d'un exercice simple : calculer la somme et le nombre des éléments pairs d'une liste. La correction de référence `01_base.py` parcourt la liste avec une boucle `for`, teste la parité avec `x % 2 == 0`, puis met à jour deux accumulateurs. Douze variantes en sont dérivées (tableau 5.3). Chacune applique une transformation identifiée, ce qui permet d'attribuer à cette transformation toute variation des scores. Les variantes sont réparties en quatre catégories :

- des **contrôles**, dont le résultat est connu à l'avance ;
- des transformations pour lesquelles une **invariance** est attendue ;
- des réécritures qui conservent la **même stratégie** de résolution ;
- des solutions fondées sur une **stratégie différente**, dont la variante `reduce` sert de témoin négatif structurel.

Les treize fichiers forment 78 paires. L'analyse qui suit porte d'abord sur les douze paires qui comparent la référence à chacune de ses variantes (tableau 5.4). Pour chaque mesure, le comportement attendu est déduit des règles présentées au chapitre précédent, puis confronté aux scores observés.

Catégorie	Variante	Transformation appliquée
Contrôles	11 Copie exacte	Copie identique de la référence
	7 Condition `== 1`	Somme des impairs au lieu des pairs (`x % 2 == 1`)
Invariance	13 Instructions inversées	Ordre des initialisations et des mises à jour inversé
	6 Opérateurs équivalents	`(x & 1) == 0` et affectations écrites sans +=
Même stratégie	4 Condition inversée	Test inversé suivi d'un `continue`
	9 `while` avec index	Boucle `while` sur un indice explicite
	12 Sortie ajoutée	Collecte supplémentaire des indices avec `enumerate`
	5 Code mort	Instructions, condition et boucle jamais exécutées
	8 Fonctions auxiliaires	Test de parité et mise à jour extraits dans deux fonctions
Stratégie différente	3 Récursion	Parcours récursif de la liste
	2 Compréhension	Compréhension de liste, puis `sum` et `len`
	10 Témoin négatif	`functools.reduce` avec une `lambda`

Table 5.3 – Variantes contrôlées dérivées de la correction de référence.

Catégorie	Variante	TSED	EIIA	NCST
Contrôles	11 Copie exacte	1,000	1,000	1,000
	7 Condition `== 1`	1,000	1,000	1,000
Invariance	13 Instructions inversées	0,944	1,000	1,000
	6 Opérateurs équivalents	0,896	0,875	1,000
Même stratégie	4 Condition inversée	0,918	0,966	0,815
	9 `while` avec index	0,767	0,671	0,897
	12 Sortie ajoutée	0,742	0,687	0,929
	5 Code mort	0,645	**0,514**	0,765
	8 Fonctions auxiliaires	0,693	0,902	0,774
Stratégie différente	3 Récursion	0,631	0,746	0,560
	2 Compréhension	0,538	0,785	0,600
	10 Témoin négatif	**0,535**	**0,871**	0,500

Table 5.4 – Scores obtenus entre la correction de référence `01_base.py` et chacune de ses douze variantes contrôlées.

Les deux contrôles se comportent comme prévu sur les trois mesures. La copie exacte obtient un score de 1, ce qui valide le fonctionnement du pipeline. La variante 7 obtient également 1 partout : les littéraux `0` et `1` sont tous deux remplacés par la catégorie `NUM` et l'opérateur == est inchangé. Aucune des trois mesures ne distingue donc la somme des pairs de la somme des impairs. Ce résultat illustre la différence entre équivalence fonctionnelle et substituabilité : les deux programmes ne calculent pas la même chose, mais ils mobilisent exactement la même démarche (parcours, test, double accumulation). Du point de vue pédagogique, le score de 1 est donc pertinent, même s'il serait faux pour une tâche de vérification fonctionnelle.

L'analyse détaillée de chaque mesure, qui confronte le comportement attendu d'après ses règles aux scores observés, figure dans le rapport complémentaire (chapitre F). Les paragraphes suivants en présentent le bilan.

5.2.1 Bilan sur les variantes contrôlées

Les trois mesures respectent les invariances qui découlent de leurs règles de normalisation, et aucune ne distingue un changement de valeur à l'intérieur d'un test (variante 7). La variante 6 n'est une invariance que pour le NCST, qui ignore les expressions : le TSED et l'EIIA distinguent volontairement une affectation augmentée (`total += x`) de sa forme explicite (`total = total + x`), ce qui explique leurs scores de 0,896 et 0,875. Elles diffèrent en revanche nettement sur les réécritures :

— le **TSED** place les stratégies différentes sous les réécritures de même stratégie, mais avec une marge très faible, et il pénalise fortement les réécritures ;

— le **NCST** est le plus robuste à ces réécritures, mais il est sensible aux changements de niveau et présente un plancher proche de 0,5 ;

— l'**EIIA** reflète la taille et le volume des programmes : il est plus sensible au code mort qu'à un changement de stratégie.

Les mesures ne sont donc pas redondantes : chacune réagit à un type de transformation différent. Ces observations justifient l'étude de leur combinaison, présentée dans la section suivante, ainsi que des améliorations possibles de la normalisation, comme la suppression du code mort, discutées dans le chapitre consacré aux limites et aux perspectives.

5.2.2 Fusion des trois approches

L'interface Streamlit permet de combiner les trois mesures par une moyenne pondérée. En l'absence de vérité terrain, aucun jeu de poids ne peut être ajusté : la fusion est donc étudiée ici avec des poids égaux, qui correspondent au réglage par défaut de l'interface. Le tableau 5.5 donne le score obtenu pour chaque variante, ainsi que celui d'une combinaison limitée au TSED et au NCST, qui écarte l'EIIA.

Avec les trois mesures, les contrôles obtiennent 1, les invariances dépassent 0,92, les réécritures de même stratégie se situent entre 0,641 et 0,900, et les stratégies différentes entre 0,635 et 0,646. Les deux groupes se chevauchent : la récursion (0,646) dépasse le code mort (0,641), et la compréhension lui est égale au millième près. L'EIIA en est la cause : il attribue des scores élevés aux stratégies différentes (0,746 à 0,871) et son plus faible score au code mort (0,514), ce qui suffit à inverser l'ordre des deux groupes.

Sans l'EIIA, l'écart entre le score le plus faible des réécritures de même stratégie (code mort, 0,705) et le plus élevé des stratégies différentes (récursion, 0,595) atteint 0,110. Les deux mesures se compensent : le NCST relève les réécritures que le TSED pénalise. Cet écart reste néanmoins inférieur

Catégorie	Variante	Moyenne des trois	Moyenne TSED + NCST
Contrôles	11 Copie exacte	1,000	1,000
	7 Condition == `1`	1,000	1,000
Invariance	13 Instructions inversées	0,981	0,972
	6 Opérateurs équivalents	0,924	0,948
Même stratégie	4 Condition inversée	0,900	0,866
	9 `while` avec index	0,778	0,832
	12 Sortie ajoutée	0,786	0,835
	5 Code mort	0,641	0,705
	8 Fonctions auxiliaires	0,790	0,734
Stratégie différente	3 Récursion	0,646	0,595
	2 Compréhension	0,641	0,569
	10 Témoin négatif	0,635	0,518

Table 5.5 – Scores pondérés à poids égaux entre la correction de référence et chacune de ses variantes.

à celui du NCST seul, dont le score le plus faible pour une même stratégie (code mort, 0,765) dépasse de 0,165 le plus élevé des stratégies différentes (compréhension, 0,600).

Aucune combinaison ne corrige la variante 7, qui obtient 1 avec chacune des mesures. Ces résultats portent sur douze paires construites autour d'un seul exercice : ils montrent l'intérêt d'une combinaison incluant le TSED et le NCST, mais ne suffisent pas à fixer des poids. Ceux-ci devront être déterminés à partir des jugements des enseignants sur le corpus complet.

5.3 Résultats sur le corpus complet

5.3.1 Sous-corpus analysé

Le test sur le corpus général s'est limité aux 99 corrections issues de GitHub, soit 4 851 paires, au lieu des 906 corrections et 409 965 paires du corpus de départ (tableau 5.1). Les balises Markdown qui encadraient ces corrections ont été retirées. Ces fichiers sont courts : 597 lignes non vides au total, 6,03 lignes en moyenne et 22 au maximum. Seuls 37 d'entre eux définissent au moins une fonction ; les 62 autres sont des scripts écrits au niveau du module, pour lesquels le TSED et l'EIIA comparent le module complet. Les 4 851 paires ont toutes été traitées sans erreur. Un fichier ne contient que des commentaires : son AST se réduit à la racine `MODULE`, et il intervient dans 97 des 192 paires dont le TSED est inférieur à 0,2.

5.3.2 Distribution des scores

Le tableau 5.6 et la figure 5.2 résument la distribution des trois scores.

Les trois mesures se distribuent de manière très différente.

— Le **TSED** est concentré : 79 % des paires se situent entre 0,3 et 0,6, et la moitié centrale tient dans un intervalle de 0,15. Seules 71 paires dépassent 0,8.

Mesure	Min	Q1	Médiane	Moyenne	Q3	Max	Écart-type	≥ 0,9	= 1
TSED	0,017	0,359	0,441	0,435	0,507	1	0,135	31	8
EIIA	0,008	0,379	0,558	0,549	0,746	1	0,225	139	8
NCST	0,069	0,375	0,500	0,520	0,636	1	0,191	231	101

Table 5.6 – Distribution des scores sur les 4 851 paires du sous-corpus GitHub. Les deux dernières colonnes donnent un nombre de paires.

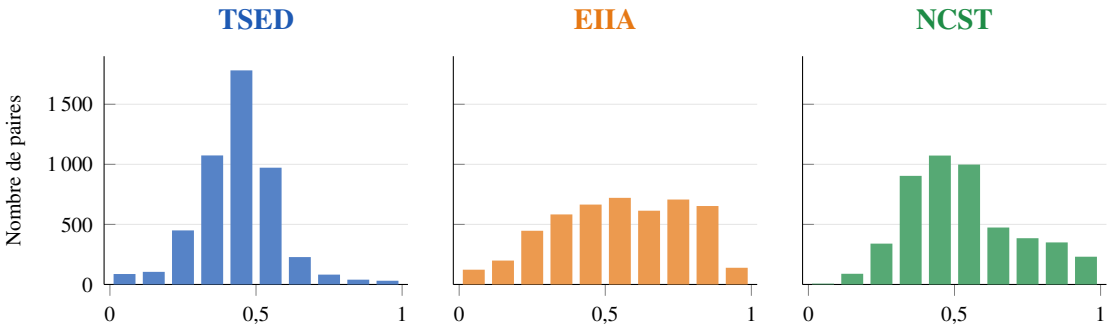


Figure 5.2 – Répartition des scores par intervalles de 0,1 sur le sous-corpus GitHub (4 851 paires).

— L'**EIIA** est la mesure la plus étalée (écart-type de 0,225) : ses valeurs couvrent presque uniformément l'intervalle de 0,3 à 0,9, et 792 paires dépassent 0,8.

— Le **NCST** prend un petit nombre de valeurs distinctes : 0,5, 0,4 et 0,667 regroupent à eux seuls 724 paires. Il attribue un score de 1 à 101 paires, dont 78 entre deux scripts. Un script court, composé d'instructions simples, se réduit à une séquence de nœuds `STATEMENT`, que le NCST ne distingue pas d'une autre séquence de même longueur.

5.3.3 Accord entre les mesures

Le tableau 5.7 donne la corrélation de rang de Spearman entre les trois scores, calculée sur l'ensemble des paires. Pour chaque mesure, les paires sont classées de la plus faible à la plus forte similarité ; la corrélation compare ensuite ces classements deux à deux. Elle vaut 1 si deux mesures ordonnent les paires exactement de la même façon et 0 si leurs classements sont sans lien. Comparer des rangs plutôt que des valeurs permet de rapprocher des mesures dont les échelles diffèrent, comme c'est le cas ici (figure 5.2).

	TSED	EIIA	NCST
TSED	1	0,763	0,364
EIIA	0,763	1	0,212
NCST	0,364	0,212	1

Table 5.7 – Corrélation de rang de Spearman entre les trois scores (4 851 paires).

Le TSED et l'EIIA classent les paires de façon proche (0,763). Les deux mesures sont calculées sur le même AST normalisé, et toutes deux font intervenir la taille des arbres : l'EIIA par la composante du nombre de nœuds, le TSED par son dénominateur $|A| + |B|$. Le NCST, qui ignore les expressions et ne retient que l'organisation en blocs, est en revanche faiblement corrélé aux deux autres (0,364 et 0,212).

L'écart entre les mesures dépend aussi de la nature des fichiers comparés (tableau 5.8).

Type de paire	Paires	TSED	EIIA	NCST
Fonctions / fonctions	666	0,486	0,533	0,570
Fonctions / script	2 294	0,418	0,565	0,417
Script / script	1 891	0,437	0,536	0,628

Table 5.8 – Score moyen selon la présence de fonctions dans les deux fichiers.

Le NCST est le plus sensible à cette distinction : sa moyenne passe de 0,628 entre deux scripts à 0,417 entre un script et un programme organisé en fonctions : les nœuds `FUNCTION` de ce dernier n'ont pas de correspondant dans un script. L'EIIA reste stable d'un type de paire à l'autre (de 0,533 à 0,565).

5.3.4 Paires candidates

Vingt-quatre paires obtiennent au moins 0,9 sur les trois mesures, dont six un score de 1 partout. Elles relient presque toujours des fichiers de numéros voisins : parmi les 31 paires dont le TSED atteint 0,9, 18 associent deux corrections consécutives. Dans les cas examinés, ces corrections consécutives traitent des variantes d'un même problème.

Ces paires correspondent au cas recherché. Les corrections 13 et 14, par exemple, ne calculent pas la même chose (la première compte les chiffres et les lettres d'une chaîne, la seconde les majuscules et les minuscules) mais suivent exactement la même démarche : un dictionnaire de compteurs, un parcours de la chaîne, une sélection à deux branches et deux affichages. Les trois mesures valent 1. La série 37 à 41 présente le même phénomène : construire la liste des carrés de 1 à 20, puis l'afficher en entier, n'en afficher que les cinq premiers ou les cinq derniers éléments, tous sauf les cinq premiers, ou sa conversion en tuple. Ces paires sont des candidates naturelles à la substitution, à confirmer par un enseignant.

5.3.5 Divergences entre les mesures

Les paires pour lesquelles les mesures divergent illustrent la dimension capturée par chacune.

NCST élevé, TSED faible. Vingt-trois paires obtiennent un NCST de 1 avec un TSED inférieur à 0,6. Les corrections 10 et 88 (TSED 0,461, NCST 1) sont deux scripts de trois instructions dont l'une est une compréhension : l'un trie les mots distincts d'une phrase, l'autre filtre les multiples de 5 et de 7. Le NCST ne voit que la même séquence d'instructions, alors que les tâches n'ont rien en commun.

EIIA élevé, TSED faible. Trente-trois paires obtiennent un EIIA d'au moins 0,9 avec un TSED inférieur à 0,5. Les corrections 5 et 13 (TSED 0,453, EIIA 0,925, NCST 0,322) en donnent un exemple : la première définit une classe à trois méthodes qui lit une chaîne et l'affiche en majuscules, la seconde compte les chiffres et les lettres d'une chaîne par un parcours et une sélection. Les deux programmes contiennent un nombre voisin de nœuds, d'affectations et d'appels, ce qui suffit à rapprocher leurs profils alors que leurs démarches n'ont rien en commun.

TSED élevé, NCST faible. Sept paires obtiennent un TSED d'au moins 0,8 avec un NCST inférieur à 0,7. Pour les corrections 26 et 95 (TSED 0,765, NCST 0,296), le TSED ne compare que les fonctions principales : une fonction qui renvoie une somme d'une part, une méthode qui renvoie une chaîne d'autre part. Le NCST, qui porte sur le fichier entier, voit en revanche les trois classes et l'héritage de la correction 95.

Code hors fonction. Inversement, les corrections 66 et 67 obtiennent 1 au TSED et à l'EIIA, mais 0,929 au NCST. Elles partagent la même fonction de Fibonacci récursive et ne diffèrent que par le code placé après elle : la première affiche un terme, la seconde construit la liste des termes par une compréhension. Seul le NCST, qui lit le fichier entier, perçoit cette différence.

Ces divergences confirment, sur des programmes réels, les comportements observés sur les variantes contrôlées. Elles relèvent pour partie du choix de l'arbre comparé par le TSED et l'EIIA, discuté dans le chapitre consacré aux limites.

5.4 Synthèse comparative

Le tableau 5.9 rassemble les observations des deux phases d'évaluation.

	TSED	EIIA	NCST
Variantes contrôlées	Sépare les deux groupes avec une marge de 0,014 ; pénalise les réécritures de même stratégie	Ne sépare pas les groupes ; score le plus faible attribué au code mort	Sépare les deux groupes avec une marge de 0,165 ; sensible aux changements de niveau
Corpus GitHub	Distribution resserrée autour de 0,45 ; peu de paires au-dessus de 0,8	Distribution étalée ; 792 paires au-dessus de 0,8	Peu de valeurs distinctes ; 101 paires à 1, surtout entre scripts
Stratégies différentes face à la référence	0,535 à 0,631	0,746 à 0,871	0,500 à 0,600
Principale faiblesse	Ne compare qu'une partie du programme lorsque les deux fichiers ont des fonctions	Mesure un volume, non une démarche	Ne distingue pas deux scripts sans structure de contrôle ayant le même nombre d'instructions
Usage recommandé	Départage les programmes que le NCST confond	Filtre de taille et de complexité	Mesure principale de proximité de démarche

Table 5.9 – Synthèse du comportement des trois mesures sur les deux phases d'évaluation.

Les deux phases ne conduisent pas au même classement des mesures. Sur les variantes contrôlées, le NCST est le plus fiable : il sépare les réécritures de même stratégie des stratégies différentes avec l'écart le plus large (0,165), là où le TSED ne dispose que d'une marge de 0,014. Sur le corpus GitHub, ce même NCST montre sa limite : les programmes y sont si courts que beaucoup se réduisent à une séquence d'instructions simples, et le NCST attribue un score de 1 à des corrections dont les tâches n'ont rien en commun. Le TSED, qui conserve les expressions, leur attribue des scores nettement inférieurs à 1 (0,461 et 0,662 pour les deux exemples cités). L'EIIA, fortement corrélé au TSED sur le

corpus et incapable de séparer les stratégies sur les variantes, apporte peu d'information propre : il rapproche des programmes de taille comparable, quelle que soit leur démarche.

Aucune mesure ne suffit donc seule, et la combinaison du TSED et du NCST est la plus prometteuse. Sur les variantes contrôlées, elle conserve un écart de 0,110 entre les deux groupes, alors que la moyenne des trois mesures les fait se chevaucher. Sur le corpus, les deux mesures sont faiblement corrélées (0,364) : elles ne classent pas les paires de la même façon. L'EIIA peut être conservé comme filtre préalable, pour écarter les paires de tailles très différentes.

Deux réserves s'appliquent à ces conclusions. Les scores du TSED et du NCST restent éloignés de 0 même entre programmes sans démarche commune : ils doivent être lus de manière relative, en classant les paires plutôt qu'en appliquant un seuil absolu. Par ailleurs, aucune vérité terrain n'est disponible : les poids de la combinaison et le seuil au-delà duquel deux exercices sont jugés substituables devront être fixés à partir du jugement des enseignants sur les paires candidates.

Chapitre 6

Discussion sur les limites et perspectives

6.1 Limites techniques des méthodes

TSED : échelle du score et choix de l'arbre comparé. Rapporter la distance à $|A| + |B|$ garantit un score compris entre 0 et 1, mais resserre l'échelle : deux arbres de même taille sans aucun label commun obtiennent au plus 0,5, et tout score est supérieur à celui que donnerait $\max(|A|, |B|)$. Un score de 0,5 peut donc correspondre à deux arbres sans aucun nœud commun, et non à une similarité de moitié. Par ailleurs, lorsque les deux fichiers contiennent des fonctions, seule la fonction principale est comparée : le code placé hors de toute fonction, comme la lecture des entrées et l'affichage du résultat, est ignoré. Lorsque l'un des fichiers n'en contient aucune, les modules complets sont comparés, avec toutes leurs fonctions. Le périmètre comparé dépend donc de la paire, et deux scores obtenus selon des règles différentes ne sont pas strictement comparables.

NCST : compréhensions et fonctions anonymes. Les nœuds `COMPREHENSION` et `LAMBDA` ne sont que des marqueurs : ils indiquent la présence de la construction, pas ce qu'elle réalise. Une compréhension avec filtre parcourt pourtant une collection et teste une condition, comme une boucle contenant une sélection. Une piste consisterait à la traduire en `ITERATION` contenant une `SELECTION`, et à conserver la structure du corps d'une `lambda`. Un essai préliminaire sur les variantes contrôlées fait alors passer le score NCST de la compréhension de 0,600 à 0,917, au niveau des réécritures de même stratégie. Ce choix reviendrait toutefois à considérer la compréhension comme la même démarche que la boucle, ce qui modifie la classification retenue pour la variante 2.

Conversion et normalisation de l'AST. Les règles de conversion et de normalisation ont elles aussi des contreparties : une construction absente de la table perd sa forme, l'*inlining* ne s'applique qu'aux fonctions réduites à un seul `return`, et les fonctions prédéfinies ou importées ne sont jamais remplacées par leur code. Ces limites sont détaillées dans le rapport complémentaire (section C.2).

6.2 Limites de l'évaluation

Les limites précédentes portent sur les méthodes elles-mêmes. L'évaluation qui en a été faite présente aussi plusieurs limites, qui restreignent la portée des conclusions du chapitre 5.

Absence de vérité terrain. Aucune paire d'exercices n'est annotée selon son degré de substituabilité. Les mesures ont donc été évaluées sur leur comportement attendu, et non sur leur capacité à reproduire un jugement pédagogique. Un score élevé signale une proximité de représentation ; il ne prouve pas que deux exercices soient interchangeables pour un apprenant. Pour la même raison, les poids de la combinaison et les seuils de décision n'ont pas pu être calibrés.

Variantes construites autour d'un seul exercice. Les douze variantes contrôlées dérivent toutes de la même correction de référence, qui associe un parcours, une sélection et deux accumulateurs. Les conclusions tirées de ces variantes valent pour ce type de programme et ne se transposent pas directement à des exercices récursifs, à des programmes orientés objet ou à des scripts sans structure de contrôle. Les variantes ont en outre été écrites manuellement, et leur classement (même stratégie ou stratégie différente) constitue une hypothèse, que seul un enseignant pourrait confirmer.

Sous-corpus restreint et peu représentatif. Le test sur le corpus général s'est limité aux 99 corrections issues de GitHub (4 851 paires), au lieu des 906 corrections du corpus de départ (409 965 paires). Ces programmes sont très courts (6,03 lignes en moyenne, contre 13,39 pour le corpus entier), et près des deux tiers sont des scripts sans fonction. Les comportements observés, comme les nombreux scores NCST égaux à 1, n'ont donc pas été vérifiés sur des corrections plus longues.

Un seul langage. Toutes les expériences portent sur Python. L'objectif du projet est pourtant de comparer des exercices quel que soit le langage ; le comportement des mesures sur d'autres langages, comme Java ou C, n'a pas été étudié.

Indicateurs d'accord, et non de validité. Les corrélations calculées sur le corpus mesurent l'accord entre les trois scores. Une forte corrélation peut traduire une dépendance commune à la taille des programmes plutôt qu'une même perception de la démarche. Ces indicateurs aident à interpréter les mesures, mais ne remplacent pas une validation externe.

6.3 Difficultés rencontrées

Définir ce que l'on mesure. La première difficulté a été conceptuelle. La notion de substituabilité n'a pas de définition opérationnelle, et elle se distingue à la fois de la similarité textuelle, de la détection de plagiat et de l'équivalence fonctionnelle. En l'absence de données annotées et de traces d'apprenants, il a fallu déterminer quelles propriétés des corrections pouvaient en être un indice raisonnable, puis accepter que les mesures produites restent des indicateurs à interpréter.

Choisir la source d'information. Les premières expériences sur les énoncés (TF–IDF et SBERT) ont donné un signal thématique, mais trop dépendant du vocabulaire et du contexte implicite des consignes. Le passage à une analyse du code des corrections a demandé de reconstruire la démarche, puis d'écarter plusieurs pistes prometteuses dans la littérature, comme les graphes de dépendances, dont le coût de comparaison est incompatible avec un corpus de plusieurs centaines d'exercices.

Qualité des données. Tree-sitter ne signale pas les erreurs de syntaxe par une exception : un fichier invalide produit un arbre partiel qui traverse tout le pipeline et reçoit des scores sans signification. Le

contrôle syntaxique ajouté a révélé que 100 des 906 corrections étaient invalides, la plupart à cause de balises Markdown héritées de leur collecte. Le corpus contient aussi des fichiers dégénérés, comme une correction réduite à des commentaires, qui pèsent sur les statistiques.

Diagnostiquer des scores inattendus. Plusieurs comportements des mesures ne sont apparus qu'à l'analyse des résultats. Sur le corpus, un cinquième des paires obtenait un TSED nul : la distance d'édition pouvait dépasser la taille du plus grand arbre, et les scripts sans fonction étaient comparés à des fonctions isolées. La correction de ces deux points a modifié l'échelle du TSED et conduit à reprendre les résultats et leur interprétation.

Coût de calcul. Le nombre de paires croît avec le carré du nombre de fichiers : les 906 corrections forment 409 965 paires, et chaque paire demande une distance d'édition d'arbres. Sur les programmes courts du sous-corpus, le calcul prend déjà de l'ordre de 0,1 seconde par paire, sans parallélisation.

6.4 Perspectives

Valider avec les enseignants. La suite la plus directe consiste à soumettre les paires candidates à des enseignants. Leurs jugements constitueraient la vérité terrain qui manque aujourd'hui : ils permettraient de vérifier si une proximité de structure correspond à une substituabilité, de fixer les poids de la combinaison et de déterminer un seuil de décision. Avec suffisamment de paires annotées, les poids pourraient être appris, par exemple par une régression logistique sur les trois scores.

Étendre l'évaluation. L'évaluation doit être reprise sur les 906 corrections, ce qui suppose de paralléliser le calcul des paires et de ne construire chaque représentation qu'une fois par fichier. De nouvelles variantes contrôlées, dérivées d'exercices d'autres natures (récursion, chaînes de caractères, programmation orientée objet), permettraient de vérifier que les conclusions ne dépendent pas de l'exercice de référence choisi.

Améliorer la normalisation. Plusieurs écarts observés sur les variantes relèvent de la normalisation et non des mesures : élimination du code mort, rapprochement des boucles `for` et `while` équivalentes, *inlining* des fonctions précédées d'affectations simples, traduction des compréhensions en parcours et sélection. Chacune de ces règles doit toutefois être pesée : plus la normalisation rapproche des écritures, plus elle risque d'effacer des différences de démarche que l'enseignant juge significatives.

Comparer le programme entier. Le TSED et l'EIIA ne comparent que la fonction principale lorsque les deux fichiers en contiennent. Une comparaison du programme entier, qui apparierait les fonctions des deux fichiers avant de sommer les distances, prendrait en compte les fonctions auxiliaires et le code placé au niveau du module. Pour le NCST, un appariement ordonné optimal, et non glouton, ainsi que la prise en compte du parent et du type d'arête, rendraient la mesure plus exigeante.

Passer à d'autres langages. Tree-sitter fournit des grammaires pour Java, C et C++, déjà installées dans le projet. Le NCST, qui ne retient que des catégories structurelles communes à ces langages, est le meilleur candidat à une comparaison inter-langages ; il faudrait notamment écrire, pour chaque langage, la table de correspondance entre ses nœuds et les catégories du NCST.

Combiner code et énoncés. Les mesures sur le code et les mesures sur les énoncés capturent des dimensions différentes : la démarche d'un côté, le thème de l'autre. Leur combinaison permettrait de distinguer deux exercices de même démarche sur des thèmes différents, utiles pour varier les entraînements, de deux exercices de même thème mais de démarches distinctes.

Intégrer les mesures dans PLaTon. À plus long terme, les mesures pourraient être intégrées à la plateforme PLaTon, sous la forme d'un outil de recherche d'exercices proches pour l'enseignant ou d'un module de recommandation pour l'étudiant. Les interactions des apprenants avec les exercices recommandés fourniraient alors les traces qui manquent aujourd'hui, et permettraient de sortir progressivement de la situation de démarrage à froid.

Conclusion générale

Ce stage s'est attaché à une question posée par les environnements d'apprentissage de la programmation : comment reconnaître, dans une banque d'exercices, deux activités qui, sans être identiques, mobilisent une démarche suffisamment proche pour être substituables ? Une telle mesure permettrait à un enseignant de retrouver des exercices voisins de ceux qu'il utilise, et à un étudiant de s'entraîner sur des problèmes comparables sans refaire la même consigne. La difficulté tient à la fois à la notion elle-même, qui ne se confond ni avec la similarité textuelle, ni avec le plagiat, ni avec l'équivalence fonctionnelle, et au contexte de démarrage à froid : aucune paire n'est annotée et aucune trace d'apprenant n'est disponible.

L'état de l'art a montré la diversité des approches de la similarité de code, des comparaisons textuelles et lexicales aux graphes de dépendances et aux modèles appris. Les expériences préliminaires sur les énoncés ont fourni un signal thématique, mais trop sensible au vocabulaire pour caractériser une démarche. Le travail s'est donc concentré sur le code des corrections de référence, analysé de manière statique, avec trois mesures complémentaires et interprétables. L'EIIA résume chaque programme par un profil de métriques structurelles. Le TSED compare les arbres syntaxiques abstraits normalisés au moyen d'une distance d'édition. Le NCST, construit au cours de ce stage, ne conserve que l'organisation des structures de contrôle et compare deux programmes par un appariement ordonné de leurs nœuds. Ces mesures sont réunies dans un pipeline unique et dans une interface qui permet de rejouer une comparaison et d'inspecter chacune des représentations intermédiaires.

L'évaluation sur des variantes contrôlées a permis de relier chaque score aux transformations qui le font varier. Les trois mesures respectent les invariances prévues par leur normalisation : la copie exacte et le changement de littéral (variante 7) obtiennent 1 avec chacune d'elles. Elles diffèrent en revanche sur les réécritures. Le NCST sépare le mieux les réécritures de même stratégie des stratégies différentes, mais il dépend du niveau d'imbrication et présente un plancher proche de 0,5. Le TSED distingue finement les programmes, mais il pénalise fortement les réécritures qui conservent la démarche. L'EIIA mesure avant tout un volume : il réagit davantage au code mort qu'à un changement de stratégie. L'analyse du sous-corpus GitHub, limitée à 99 des 906 corrections, a retrouvé ces comportements sur des programmes réels et en a révélé d'autres : sur des scripts très courts, le NCST ne distingue plus des programmes dont les tâches n'ont rien en commun, alors que le TSED les départage.

La principale conclusion est donc qu'aucune mesure ne suffit seule. La combinaison du TSED et du NCST, faiblement corrélés et sensibles à des transformations différentes, est la plus prometteuse ; l'EIIA, fortement lié à la taille des programmes, trouve plutôt sa place comme filtre préalable. Les scores doivent être lus de manière relative, pour classer des paires, et non comme des pourcentages de similarité. Le pipeline produit ainsi, sans données d'apprenants, des indicateurs continus et explicables qui permettent de désigner des paires candidates à la substitution. Les paires les mieux classées du corpus, comme deux corrections qui comptent des caractères de natures différentes avec la même démarche, illustrent le type de rapprochement recherché.

Ces résultats restent à confirmer. Ils reposent sur des variantes dérivées d'un seul exercice, sur

un sous-corpus de programmes courts et sur un seul langage, et aucune vérité terrain ne permet encore d'affirmer qu'une proximité de structure correspond à une substituabilité pédagogique. La suite naturelle de ce travail est la validation des paires candidates par des enseignants, qui permettra de calibrer la combinaison des mesures, puis l'extension de l'évaluation au corpus complet, à d'autres langages et à des normalisations plus riches.

Bilan personnel

Sur le plan personnel, ce stage a été ma première expérience d'un travail de recherche. Contrairement à un projet de développement dont le résultat est fixé à l'avance, il a fallu définir ce que l'on cherchait à mesurer, essayer des pistes, en abandonner plusieurs et accepter que certains résultats remettent en cause des choix déjà faits. J'ai appris à construire une démarche expérimentale : formuler ce qu'une mesure devrait produire avant de la calculer, concevoir des cas de test dont le résultat est connu, puis expliquer chaque écart entre l'attendu et l'observé.

Sur le plan technique, j'ai approfondi l'analyse syntaxique de programmes avec Tree-sitter, la manipulation d'arbres syntaxiques, les distances d'édition entre arbres et les méthodes de traitement du langage naturel (TF–IDF, SBERT). La réalisation de l'état de l'art m'a appris à rechercher, lire et comparer des articles scientifiques, et à les citer avec rigueur. La rédaction de ce rapport en LaTeX m'a enfin demandé de rendre des choix techniques compréhensibles pour un lecteur non spécialiste.

J'ai également gagné en autonomie et en organisation. Les points réguliers avec mon maître de stage m'ont appris à présenter un travail en cours, à justifier mes choix et à tenir compte des remarques pour réorienter le travail. **[À vérifier :** Ce stage a confirmé mon intérêt pour [domaine visé : par exemple le traitement de données, l'analyse de code ou les technologies éducatives], que je souhaite poursuivre [dans mon projet professionnel : poste, poursuite d'études, etc.].**]**

Bibliographie

[1] Uri Alon, Meital Zilberstein, Omer Levy et Eran Yahav. « code2vec : Learning Distributed Representations of Code ». In : *Proceedings of the ACM on Programming Languages (POPL)* (2019).

[2] I.D. Baxter et al. « Clone detection using abstract syntax trees ». In : *Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272)*. Proceedings. International Conference on Software Maintenance. Bethesda, MD, USA : IEEE Comput. Soc, 1998, p. 368-377. isbn : 978-0-8186-8779-2. doi : 10.1109/ICSM.1998.738528. url : http://ieeexplore.ieee.org/document/738528/.

[3] Alisson Antônio De Oliveira. « Assessing Programming Difficulty and Effort : Statistical Correlations with the Index of Internal Effort ». In : *Anais da XII Escola Regional de Informática de Goiás (ERI-GO 2024)*. Escola Regional de Informática de Goiás. Brasil : Sociedade Brasileira de Computação, 5 déc. 2024, p. 21-30. doi : 10.5753/erigo.2024.4795. url : https://sol.sbc.org.br/index.php/erigo/article/view/32208.

[4] Zhangyin Feng, Daya Guo, Duyu Tang et al. « CodeBERT : A Pre-Trained Model for Programming and Natural Languages ». In : *Findings of EMNLP*. 2020, p. 1536-1547.

[5] Daya Guo, Shuo Ren, Shuai Lu et al. « GraphCodeBERT : Pre-training Code Representations with Data Flow ». In : *ICLR*. 2021.

[6] Keith J. Holyoak et Kyunghee Koh. « Surface and structural similarity in analogical transfer ». In : *Memory & Cognition* 15.4 (juill. 1987), p. 332-340. issn : 0090-502X, 1532-5946. doi : 10.3758/BF03197035. url : http://link.springer.com/10.3758/BF03197035.

[7] Roya Hosseini et Peter Brusilovsky. « A study of concept-based similarity approaches for recommending program examples ». In : *New Review of Hypermedia and Multimedia* 23.3 (2017), p. 161-188.

[8] Sébastien Jolivet, Badmavasan Kirouchenassamy, Amel Yessad et Vanda Luengo. « Modélisation et décision de rétroactions épistémiques dans l'environnement AlgoPython ». In : *Environnements Informatiques pour l'Apprentissage Humain*. Lille, France : Université de Lille, juin 2025. url : https://hal.science/hal-05163937.

[9] Qi Liu et al. « Finding Similar Exercises in Online Education Systems ». In : *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2018, p. 1821-1830.

[10] Yiyang Lu et al. *CSSG : Measuring Code Similarity with Semantic Graphs*. 14 jan. 2026. doi : 10.48550/arXiv.2601.04085. arXiv : 2601.04085[cs.PL]. url : http://arxiv.org/abs/2601.04085.

[11] Marcus Messer, Neil C. C. Brown, Michael Kölling et Miaojing Shi. « Automated Grading and Feedback Tools for Programming Education : A Systematic Review ». In : *ACM Transactions on Computing Education* 24.1 (2024).

[12] Dragana Milovancevic. « Automated Induction for Proving Program Equivalence ». In : ().

[13] Dragana Milovančević et al. « Formal Autograding in a Classroom ». In : *Programming Languages and Systems*. Sous la dir. de Viktor Vafeiadis. T. 15695. Series Title : Lecture Notes in Computer Science. Cham : Springer Nature Switzerland, 2025, p. 154-174. isbn : 978-3-031-91120-0. doi : 10.1007/978-3-031-91121-7_7. url : https://link.springer.com/10.1007/978-3-031-91121-7_7.

[14] Jie Mu et al. « BCRNet : Bidirectional contrastive representation network for deep multimodal learning of exercise representations ». In : *Neurocomputing* 548 (2023), p. 126409.

[15] José Paiva, José Leal et Álvaro Figueira. « Comparing semantic graph representations of source code : The case of automatic feedback on programming assignments ». In : *Computer Science and Information Systems* 21.1 (2024), p. 117-142. issn : 1820-0214, 2406-1018. doi : 10.2298/CSIS230615004P. url : https://doiserbia.nb.rs/Article.aspx?ID=1820-02142400004P.

[16] Radek Pelánek. « Measuring Similarity of Educational Items : An Overview ». In : *IEEE Transactions on Learning Technologies* 13.2 (2020), p. 354-366.

[17] Lutz Prechelt, Guido Malpohl et Michael Philippsen. « Finding Plagiarisms among a Set of Programs with JPlag ». In : ().

[18] Manda Uday Kiran Reddy, Shreyas Pujar et Srujana Inturi. « Graph-Based Semantic Similarity for Programming Exercise Assessment : A Survey ». In : *2025 5th International Conference on Intelligent Technologies (CONIT)*. 2025 5th International Conference on Intelligent Technologies (CONIT). Juin 2025, p. 1-8. doi : 10.1109/CONIT65521.2025.11167193. url : https://ieeexplore.ieee.org/abstract/document/11167193.

[19] Nils Reimers et Iryna Gurevych. « Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks ». In : *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Hong Kong, China : Association for Computational Linguistics, 2019, p. 3982-3992. doi : 10.18653/v1/D19-1410. url : https://www.aclweb.org/anthology/D19-1410.

[20] Yewei Song et al. « Revisiting Code Similarity Evaluation with Abstract Syntax Tree Edit Distance ». In : *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2 : Short Papers)*. 2024, p. 38-46. doi : 10.18653/v1/2024.acl-short.3. arXiv : 2404.08817[cs.CL]. url : http://arxiv.org/abs/2404.08817.

[21] Zejie Tian, Brendan Flanagan, Yiling Dai et Hiroaki Ogata. « Automated Matching of Exercises with Knowledge Components ». In : *Proceedings of the 30th International Conference on Computers in Education (ICCE)*. 2022, p. 24-33.

[22] Milena Vujošević-Janičić, Mladen Nikolić, Dušan Tošić et Viktor Kuncak. « Software verification and graph similarity for automated evaluation of students' assignments ». In : *Information and Software Technology* 55.6 (juin 2013), p. 1004-1016. issn : 09505849. doi : 10.1016/j.infsof.2012.12.005. url : https://linkinghub.elsevier.com/retrieve/pii/S0950584912002406.

[23] Tiantian Wang, Kechao Wang, Xiaohong Su et Peijun Ma. « Detection of semantically similar code ». In : *Frontiers of Computer Science* 8.6 (1er déc. 2014), p. 996-1011. issn : 2095-2236. doi : 10.1007/s11704-014-3430-1. url : https://doi.org/10.1007/s11704-014-3430-1.

[24] Chengguan Xiang, Ying Wang, Qiyun Zhou et Zhen Yu. « Graph semantic similarity-based automatic assessment for programming exercises ». In : *Scientific Reports* 14.1 (8 mai 2024), p. 10530. issn : 2045-2322. doi : 10.1038/s41598-024-61219-8. url : https://www.nature.com/articles/s41598-024-61219-8.

[25] Kaizhong Zhang et Dennis Shasha. « Simple Fast Algorithms for the Editing Distance between Trees and Related Problems ». In : *SIAM Journal on Computing* 18.6 (déc. 1989), p. 1245-1262. issn : 0097-5397, 1095-7111. doi : 10.1137/0218082. url : http://epubs.siam.org/doi/10.1137/0218082.

[26] Adel Ziani. *Détection automatique d'exercices de programmation similaires*. Rapport de stage de M1 informatique, laboratoire Dicen, Université Gustave Eiffel. 2025.

Webographie

[27] Dicen. *Organisation du laboratoire Dicen*. url : https://dicen.org/organisation/ (visité le 24/09/2026).

[28] Dicen-IDF. *Le laboratoire DICEN-IdF (EA 7339)*. url : https://www.dicen-idf.org/equipe/ (visité le 24/09/2026).

[29] Journal officiel de la République française. *Décret n° 2019-1360 du 13 décembre 2019 portant création de l'Université Gustave Eiffel et approbation de ses statuts*. url : https://www.univ-gustave-eiffel.fr/fileadmin/UniversiteGustaveEiffel/Universite/Decret2019_1360_du_13_decembre_2019_creation_Universite_Gustave_Eiffel_et_approbation_de_ses_statuts.pdf (visité le 24/09/2026).

[30] Wikipédia. *Université Gustave-Eiffel*. url : https://fr.wikipedia.org/wiki/Universit%C3%A9_Gustave-Eiffel (visité le 24/09/2026).